

MULTI-STREAM CONVOLUTIONAL NEURAL NETWORKS FOR FALL DETECTION ON
THE KFALL DATASET USING CONTINUOUS WAVELET TRANSFORM SCALOGRAMS

by

Jaydan Herrera

Submitted in partial fulfillment to the requirements for the degree of Bachelor of Science in
Computer Science

Department of Computer and Mathematical Sciences
New Mexico Highlands University

May 2026

Contents

1 Introduction.....	3
2 Background.....	4
2.1 Inertial Measurement Units.....	4
2.2 Time Frequency and Spectrograms.....	4
2.3 Continuous Wavelet Transform and Scalograms.....	5
2.4 Convolutional Neural Networks	6
2.4.1 Digital Image Representation	6
2.4.2 Architecture.....	8
2.4.3 Convolution Layer	8
2.4.4 Pooling Layer.....	11
2.4.5 Flatten Layer	12
2.4.6 Densely Connected Layers	13
2.4.7 Output Layer	14
2.5 Multi-Stream Architectures.....	14
3 Data	15
3.1 Collection.....	15
3.2 Exploration.....	16
4 Methodology	19
4.1 Data Preprocessing.....	19
4.2 Feature Extraction	19
4.3 Model Selection	20
4.4 Model Training.....	21
4.5 Model Evaluation.....	22
5 Results.....	23
6 Discussion	27
7 Conclusion	28

1 Introduction

Falls are a significant public health concern among older adults. According to the Centers for Disease Control and Prevention, “Falls are the leading cause of injury for adults ages 65 years and older. Over 14 million, or 1 in 4 older adults report falling every year”. Falls can lead to serious injuries, including fractures that may lead to long term disability. Additionally, the fear of falling can reduce physical activity and negatively affect overall health (National Institute on Aging). Consequently, there is a need for effective fall detection systems. Fall detection systems are generally categorized into context-aware systems and wearable systems. Context-aware systems are typically limited to indoor environments, whereas wearable systems can operate across a variety of settings. Wearable systems may focus on pre-impact fall detection, which aims to predict falls before they occur, or post-fall detection, which aims to identify fall events promptly and trigger timely medical alerts (Yu et al. 2). “The majority of existing studies focus on post-fall detection, which is designed to rapidly detect fall events and initiate medical alarms timely to reduce the frequency and severity of long lies” (qtd. in Yu et al. 2). A post-fall detection approach is employed in this study, with a focus on exclusively developing a machine learning (ML) pipeline capable of detecting fall impacts. Concentrating on the pipeline allows for optimization of detection accuracy in a controlled environment, providing a foundation for future integration with various hardware platforms. The pipeline processes accelerometer and gyroscope sensor data from the KFall dataset, extracts time–frequency features using the continuous wavelet transform (CWT), and trains a multi-stream convolutional neural network (CNN) to classify falls accurately.

2 Background

2.1 Inertial Measurement Units

An inertial measurement unit (IMU) is an "electromechanical or solid-state device that contains an array of sensors that are able to measure motion" ("Inertial Measurement Unit (IMU) - an Introduction"). IMUs measure motion by translating inertial forces into data that describes the movement of an object. The inertial sensors within an IMU include accelerometers, gyroscopes, and sometimes magnetometers. Accelerometers measure linear acceleration along the x, y, and z axes, while gyroscopes measure angular velocity along each axis. Magnetometers measure the strength and direction of the Earth's magnetic field. Micro-electromechanical systems (MEMS) technology is commonly used in modern IMUs. MEMS are miniature devices, typically measured in micrometers, that integrate electrical and mechanical components to perform specialized functions. Common examples include MEMS accelerometers, MEMS gyroscopes, and MEMS magnetometers. The advantages of MEMS technology include reliability, cost-effectiveness, low power consumption, and compact size. Some applications of IMU sensor data are motion tracking, vibration monitoring, consumer electronics, and pointing applications. ("Inertial Measurement Unit (IMU) - an Introduction"). When a person falls, their velocity, acceleration, and orientation may change significantly. Since IMUs contain accelerometers and gyroscopes, they can measure motion data along multiple axes, which can be analyzed for patterns associated with falls. Public datasets containing IMU data are therefore an important resource for fall detection research. These datasets can support the development of machine learning algorithms used for fall detection with some transformation of the raw data.

2.2 Time Frequency and Spectrograms

A signal is a function of one or more variables whose output values are quantities associated with a physical phenomenon. Examples of signals include sound waves, radio signals, stock market trends, and digital images. Many signals are represented in the time or spatial domains, which describe how the signal varies over time or space. While these representations are useful, it is often important to analyze the frequency content of a signal. A signal whose frequency content remains constant over time is called a stationary signal, whereas a signal whose frequency content changes over time is called a non-stationary signal. Non-stationary signals are common in real world applications. One of the tools used to analyze the frequency content of a signal is the Fourier transform. The Fourier transform is an integral transform that maps a function from the time or spatial domain to a complex-valued function in the frequency domain. The Fourier transform of a function f is defined as

$$F(\xi) = \int_{-\infty}^{\infty} f(x)e^{-i2\pi\xi x} dx. \quad (1)$$

F is a complex-valued function representing the frequency domain representation of the signal, ξ denotes frequency, f denotes the signal in the time or spatial domain, x denotes time or space, and i is the imaginary unit defined to be equal to $\sqrt{-1}$. The Fourier transform reveals the frequency components present in a signal, but it does not indicate when those frequencies occur in time. The short-time Fourier Transform (STFT) is used to analyze how the frequency content of a signal changes over time. By providing information about both time and frequency, it is well suited for analyzing nonstationary signals. The STFT analyzes a signal by dividing it into short,

often overlapping, segments of equal length using a window function and computing the Fourier transform of each segment. This produces a time–frequency representation that describes how the frequency content of the signal changes over time. The time–frequency representation is commonly visualized using a spectrogram. The STFT of a function f is defined as

$$X(\tau, \xi) = \int_{-\infty}^{\infty} x(t)w(t - \tau)e^{-i2\pi\xi t} dt. \quad (2)$$

The function $x(t)$ denotes the signal in the time domain, and t denotes time. The function $w(t)$ is used to localize the analysis in time. The parameter τ specifies the time at which the window is centered. Thus, $w(t - \tau)$ represents the window shifted to the time τ . The variable ξ denotes frequency, and i is the imaginary unit defined to be equal to $\sqrt{-1}$. The resulting function $X(\tau, \xi)$ is a complex-valued function representing the time-frequency domain representation of the signal. The time frequency representation of the signal given by the STFT is commonly visualized using a spectrogram, in which the horizontal axis represents time, the vertical axis represents frequency, and the color intensity represents the squared magnitude of the STFT, $|X(\tau, \xi)|^2$. When analyzing a spectrogram, both time and frequency resolution are important. Time resolution refers to the ability to localize frequency content in time, while frequency resolution refers to the ability to distinguish frequency components of the signal. Since the STFT uses a fixed window size, the time and frequency resolutions are constant across all frequencies. This creates a trade-off in which a shorter window provides good time resolution but poor frequency resolution, while a longer window provides good frequency resolution but poor time resolution. The fixed window size thus imposes a fixed trade-off between time and frequency resolution.

2.3 Continuous Wavelet Transform and Scalograms

The CWT is a mathematical technique used to analyze time-varying frequency content of non-stationary signals. The CWT does this by comparing the signal with translated and scaled versions of a wavelet, a localized oscillating function with zero mean. Since the CWT uses scaled versions of a wavelet, the width of the wavelet changes with scale. Large scales stretch the wavelet, producing a wide window that achieves high frequency resolution for low-frequency components of a signal. Conversely, small scales compress a wavelet, producing a narrow window that achieves high time resolution for high frequency components of a signal. Unlike the fixed window size used by the STFT, the scaled versions of a wavelet used in the CWT allows the CWT to adapt its time and frequency resolution, making it better suited for analyzing non-stationary signals. The CWT of a function $x(t)$ is defined as

$$X_w(a, b) = \frac{1}{\sqrt{|a|}} \int_{-\infty}^{\infty} x(t)\bar{\psi}\left(\frac{t - b}{a}\right) dt. \quad (3)$$

The function $x(t)$ denotes the signal in the time domain and t denotes time. The function $\psi(t)$ is called the mother wavelet, and $\bar{\psi}$ is the complex conjugate of ψ . The parameter a is the scaling parameter, which controls the width of the wavelet, while b is the translation parameter, which determines the position of the wavelet in time. The resulting function $X_w(a, b)$ is a complex-valued function that describes how well the signal matches a wavelet at a particular scale and time. The time-scale representation produced by the CWT is visualized using a scalogram, in

which the horizontal axis represents time, the vertical axis represents scale, and the color intensity represents magnitude of the CWT. Falls can be measured using data from an IMU, which records both linear acceleration using an accelerometer and angular velocity using a gyroscope. From these measurements, each fall can be represented by six signals. The first three signals measure acceleration along the x, y, and z axes, respectively. The next three signals measure angular velocity along the x, y, and z axes, respectively. Falls typically involve a sequence of rapidly changing events, including loss of balance, downward motion towards the ground, impact, and subsequent stillness. The frequency content of the six signals varies over time, making them non-stationary signals. Since the CWT can adapt its time and frequency resolution, it is well suited for analyzing the non-stationary signals that represent a fall.

2.4 Convolutional Neural Networks

Fully connected multilayer perceptrons are not well suited for image data because they lack translation invariance and are prone to overfitting due to the large number of trainable parameters required for high-dimensional inputs (Kromydas). These limitations motivate the use of architectures that more effectively capture the spatial structure of images. A CNN is a specialized neural network designed to process data with a grid-like structure. Since images can be represented as matrices of pixel values, they naturally possess this structure. CNNs have achieved strong performance in tasks including image classification, object detection, and segmentation. Consequently, CNNs are well suited for image processing tasks.

2.4.1 Digital Image Representation

Before discussing the details of CNNs, it is useful to first examine the data they process: digital images. Understanding the mathematical representation of images provides the foundation for the operations performed by CNNs. Digital images are commonly classified as either grayscale images or color images represented in the Red, Green, Blue (RGB) color model. A grayscale image can be represented as a two-dimensional matrix of pixel intensities, with dimensions corresponding to the image's height and width. In an 8-bit representation, the pixel intensities can be any value in the interval $[0,255]$. Given a height $H \in \mathbb{Z}^+$ and a width $W \in \mathbb{Z}^+$ a grayscale image can be mathematically represented by the function

$$I: \{0,1, \dots, H-1\} \times \{0,1, \dots, W-1\} \rightarrow [0,255].$$

A pixel value in the two-dimensional matrix that represents a grayscale image can be represented by $I(x,y)$ where x and y are the spatial coordinates of the pixel. Figure 1 shows a grayscale image with a height of 3 pixels and a width of 3 pixels. Within each cell of the matrix are the pixel values written in functional notation with the corresponding spatial coordinate inputs.

$I(0,0)$	$I(0,1)$	$I(0,2)$
$I(1,0)$	$I(1,1)$	$I(1,2)$
$I(2,0)$	$I(2,1)$	$I(2,2)$

Figure 1: Grayscale Image Matrix Representation

An RGB image can be represented as a three-dimensional array whose axes correspond to the image's height, width, and color channels. Equivalently, this array may be viewed as consisting of three two-dimensional matrices, with dimensions corresponding to the image's height and width. Each matrix represents a grayscale intensity map corresponding to one of the three color channels. The first matrix corresponds to the red channel, the second to the green channel, and the third to the blue channel. In an 8-bit representation, the entries of each matrix lie in the interval $[0,255]$, where larger values indicate greater intensity of the corresponding color component. The combination of the three channel intensities at each pixel determines the final perceived color through additive color mixing. Given a height $M \in \mathbb{Z}^+$ and a width $N \in \mathbb{Z}^+$ each color channel of an RGB image can be mathematically represented by the functions

$$R: \{0,1, \dots, M-1\} \times \{0,1, \dots, N-1\} \rightarrow [0,255],$$

$$G: \{0,1, \dots, M-1\} \times \{0,1, \dots, N-1\} \rightarrow [0,255],$$

$$B: \{0,1, \dots, M-1\} \times \{0,1, \dots, N-1\} \rightarrow [0,255].$$

An RGB image can be mathematically represented by the function

$$C: \{0,1, \dots, M-1\} \times \{0,1, \dots, N-1\} \rightarrow [0,255]^3$$

$$\text{defined as } C(x, y) = (R(x, y), G(x, y), B(x, y)).$$

A pixel value in the three-dimensional array that represents an RGB image can be represented by $C(x, y)$ where x and y are the spatial coordinates of the pixel. $C(x, y) = (R(x, y), G(x, y), B(x, y))$ so $(R(x, y), G(x, y), B(x, y))$ also represents the pixel value in the RGB image where x and y are spatial coordinates. $R(x, y)$, $G(x, y)$, and $B(x, y)$ represent the intensities of the red, green, and blue channels, respectively, at location (x, y) . Each channel can be viewed as a matrix whose entries correspond to color intensities at specific spatial coordinates.

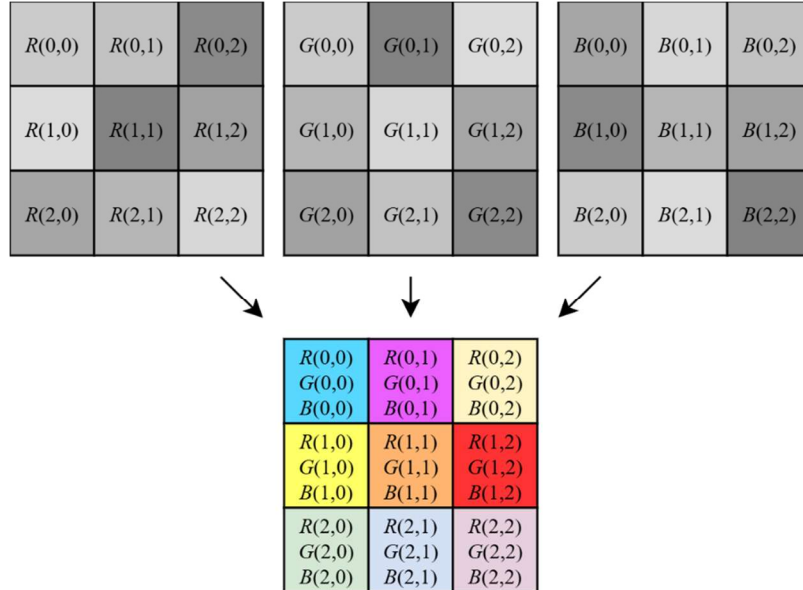


Figure 2: RGB Image Multidimensional Array Representation

Figure 2 illustrates the three channels of an RGB image and how they combine through additive color mixing to form the full RGB image. The RGB image and each of its individual channels have a height of 3 pixels and a width of 3 pixels. Pixel values in each channel are expressed in functional notation. Likewise, each component of the ordered triple representing a given pixel in the RGB image is written in functional notation.

2.4.2 Architecture

To understand how a CNN functions, it is important to examine each of its core components. These include the convolutional layer, pooling layer, flatten layer, densely connected layers, and the output layer, as shown in Figure 3.

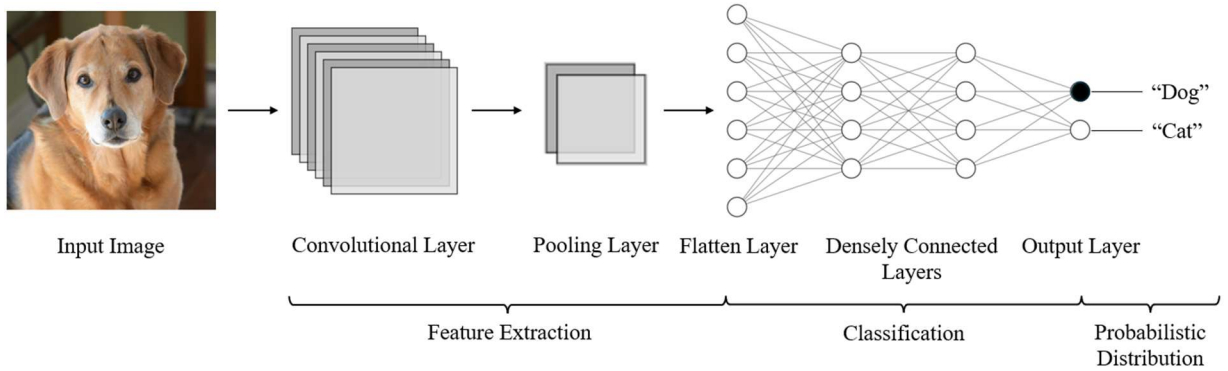


Figure 3: CNN Components. Dog image from Kerns, Nancy, “The Dogs We Love: Too Little Time Together”.

The convolutional and pooling layers form the feature extraction portion of the CNN, while the flatten layer, densely connected layers, and output layer make up the classification portion. The feature extraction stage identifies spatial patterns within the input image, and the classification stage uses these extracted features to generate predictions based on what was learned during feature extraction.

2.4.3 Convolution Layer

The mathematical foundation of the convolution layer is the convolution operation. Convolution is an integral transform that produces a function from two given functions. The convolution of two functions f and g is defined as

$$(f * g)(t) = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau. \quad (4)$$

In equation 4, the function f is multiplied by a shifted and reflected version of the function g , denoted by $g(t - \tau)$. Specifically, $g(t - \tau)$ is obtained by reflecting g about the vertical axis and then shifting g horizontally by t units. If $t > 0$, g is shifted to the right, and if $t < 0$, g is shifted to the left. Intuitively, the convolution measures the amount of overlap between f and a reflected, shifted version of g . While Equation 4 applies to continuous functions, CNNs typically process images, which are discrete. Consequently, a discrete version of the convolution operation is used in practice.

A convolutional layer extracts spatial features from an image using learnable filters called kernels. In this layer, the input image is convolved with a kernel to produce a feature map. Recall

from Section 2.4.1 that a grayscale image can be represented as a two-dimensional matrix of pixel intensities, with dimensions corresponding to the image's height and width. Similarly, a kernel for a grayscale image can be represented as a two-dimensional matrix of weight values. Each kernel has an associated bias term. The weights and bias values are learned during training and are adjusted through backpropagation to minimize the loss function and improve the network's predictive accuracy.

During the convolution operation, the kernel slides over the image, and at each spatial location where the kernel overlaps the image, the dot product between the kernel weights and the corresponding pixel values is computed. The bias term associated with the kernel is then added to this result. The sum is subsequently passed through an activation function, and the outputs of this process form a two-dimensional matrix called a feature map.

A convolutional layer is characterized by several hyperparameters, including the kernel size, number of kernels, stride, and padding. The kernel size refers to the spatial dimensions of the kernel. The number of kernels determines how many feature maps are produced, as each kernel generates one feature map. The stride specifies the number of pixels the kernel moves between successive positions. Padding refers to the addition of pixels around the border of the input image to control the spatial dimensions of the feature maps. A common padding technique is zero-padding, in which pixels with value 0 are added around the image boundary.

To compute the spatial dimensions of a feature map, the following formulas can be used to determine its height and width, respectively:

$$F_h = \left\lfloor \frac{I_h + 2P - K_h}{S} \right\rfloor + 1,$$
$$F_w = \left\lfloor \frac{I_w + 2P - K_w}{S} \right\rfloor + 1.$$

I_h is the input image's height, I_w is the input image's width, K_h is the kernel's height, K_w is the kernel's width, P is the padding, and S is the stride.

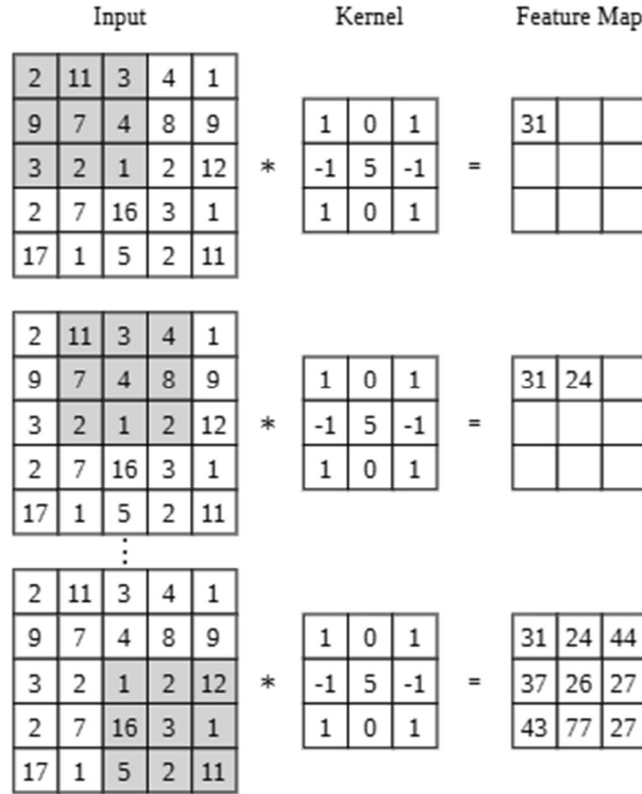


Figure 4: Convolution of Grayscale Image

In Figure 4 an example of a convolution between a grayscale image and a kernel is provided. The image has a height of 5 pixels and a width of 5 pixels. The kernel has a height of 3 pixels and a width of 3 pixels. Figure 4 shows the convolution of the image with the kernel at the first, second, and last position where the kernel overlaps the image. In this example there is one kernel, the bias term associated with the kernel is 0, the activation function is the Rectified Linear Unit (RELU) function, the stride is 1, and there is no padding. The RELU activation function is defined as:

$$f: \mathbb{R} \rightarrow \mathbb{R}$$

$$f(x) = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

The RELU function takes a real number as input and outputs the real number if it is greater than 0, and outputs 0 if the real number is less than or equal to 0. The height of the feature map, F_h , is $\left\lfloor \frac{5+2(0)-3}{1} \right\rfloor + 1 = \lfloor 2 \rfloor + 1 = 2 + 1 = 3$. The width of the feature map, F_w , is $\left\lfloor \frac{5+2(0)-3}{1} \right\rfloor + 1 = \lfloor 2 \rfloor + 1 = 2 + 1 = 3$.

The convolutional layer for an RGB image differs slightly from that of a grayscale image. As discussed in Section 2.4.1, an RGB image can be represented as a three-dimensional array, with axes corresponding to the image's height, width, and color channels. Accordingly, the kernels in the convolutional layer also have three channels. During convolution, each channel of the kernel is applied to the corresponding channel of the input image. At each spatial location where the kernel channels overlap the corresponding channels of the image, the dot product between the

kernel weights and the corresponding pixel values is computed separately for each channel. This produces three values, one per channel, which are then summed. The bias term associated with the kernel is added to this sum, and the result is passed through an activation function. The result of this process is a two-dimensional feature map.

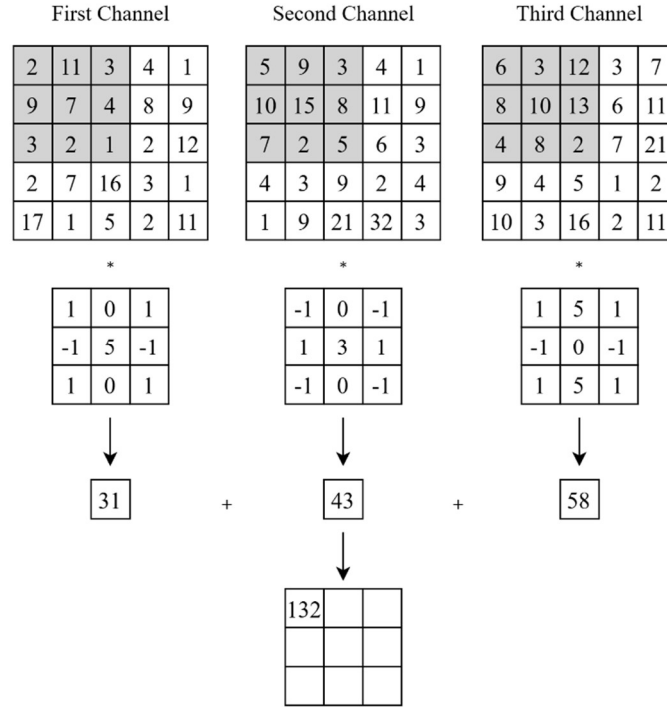


Figure 5: Convolution of RGB Image

Figure 5 illustrates the convolution of an RGB image and a kernel at the first spatial location where the kernel channels overlap the corresponding image channels. The RGB image has a height of 5 pixels, a width of 5 pixels, and 3 channels. The kernel has a height of 3 pixels, a width of 3 pixels, and 3 channels. The bias term associated with the kernel is 0. The activation function used is the RELU function. The feature map has a height, F_h of $\left\lfloor \frac{5+2(0)-3}{1} \right\rfloor + 1 = [2] + 1 = 2 + 1 = 3$. The feature map has a width, F_w , of $\left\lfloor \frac{5+2(0)-3}{1} \right\rfloor + 1 = [2] + 1 = 2 + 1 = 3$.

2.4.4 Pooling Layer

A pooling layer reduces the spatial dimensions of feature maps while preserving the extracted features. The two most common types of pooling are max pooling and average pooling. In max pooling, a filter slides across the feature map and selects the maximum value within each local region where the filter overlaps the feature map. In average pooling, the filter instead computes the average of the values within each local region where the filter overlaps the feature map. In both cases, the result is a feature map with reduced dimensions.

The hyperparameters of a pooling layer are the filter size, which determines the spatial dimensions of the pooling region, and the stride, which specifies how far the filter moves between successive positions. A convolution layer and pooling layer form a convolutional block. There can be multiple convolutional blocks stacked within a CNN.

To compute the spatial dimensions of a feature map after pooling, the following formulas can be used to determine its height and width, respectively:

$$P_h = \left\lfloor \frac{F_h - f_h}{s} \right\rfloor + 1,$$

$$P_w = \left\lfloor \frac{F_w - f_w}{s} \right\rfloor + 1.$$

F_h is the feature map's height, F_w is the feature map's width, f_h is the filter's height, f_w is the filter's width, and s is the stride.

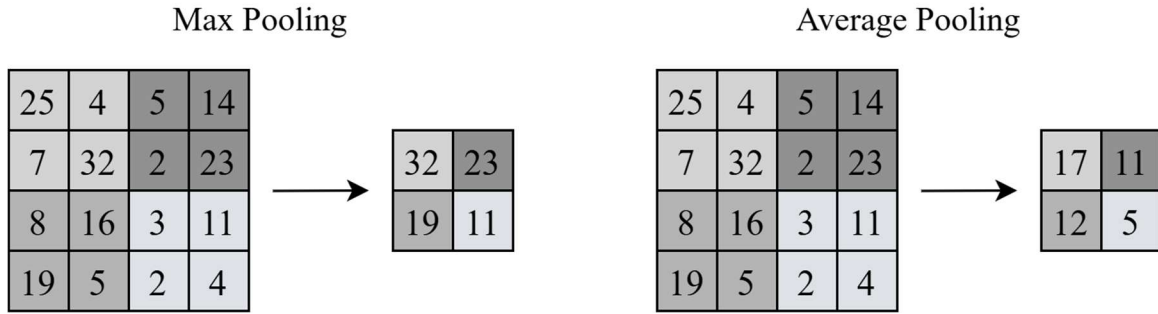


Figure 6: Max Pooling and Average Pooling Examples

Figure 6 provides examples of both max pooling and average pooling. In both cases, the feature map dimensions, pooling filter size, and stride are identical. The input feature maps have a height and width of 4 pixels, the pooling filters have a height and width of 2 pixels, and the stride is 2. Since these parameters are the same in both examples, the spatial dimensions of the feature maps after pooling are identical. The height of the feature maps after pooling, P_h , is $\left\lfloor \frac{4-2}{2} \right\rfloor + 1 = \lfloor 1 \rfloor + 1 = 1 + 1 = 2$. The width of the feature maps after pooling, P_w , is $\left\lfloor \frac{4-2}{2} \right\rfloor + 1 = \lfloor 1 \rfloor + 1 = 1 + 1 = 2$.

2.4.5 Flatten Layer

The flattening layer follows the convolutional blocks and converts the feature maps into a 1D feature vector. The values of the 1D feature vector will be fed to the densely connected layers. Figure 7 shows an example of two feature maps being converted into a 1D feature vector.

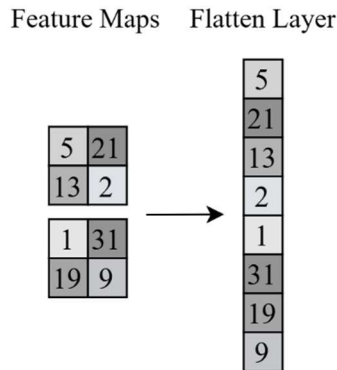


Figure 7: Flatten Layer Example

2.4.6 Densely Connected Layers

Densely connected layers follow the flatten layer in a CNN and are used for classification. These layers are based on the perceptron, the simplest form of a neural network, illustrated in Figure 8.

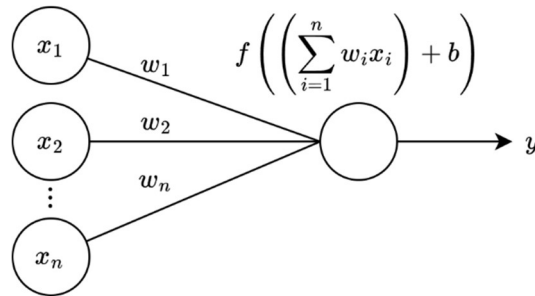


Figure 8: Perceptron

A perceptron consists of a neuron, which is a computational element that receives inputs, computes a weighted sum, adds a bias term, and inputs the result into an activation function to produce an output. The inputs to the neuron, shown in Figure 8 $x_1, x_2, \dots, x_n$, are each associated with a weight, $w_1, w_2, \dots, w_n$. In the neuron, each input x_i is multiplied by its associated weight w_i , and the sum of these products is combined with the bias. Then, the activation function transforms this sum to produce the neuron's output, labeled y in Figure 8. A perceptron is unable to learn complex functions and as a result performs poorly on non-linear problems. As a result, a more sophisticated neural network is needed.

A multi-layer perceptron, shown in Figure 9, consists of an input layer, one or more hidden layers, and an output layer. Since a multi-layer perceptron has more layers than a perceptron it can learn complex functions and solve non-linear problems.

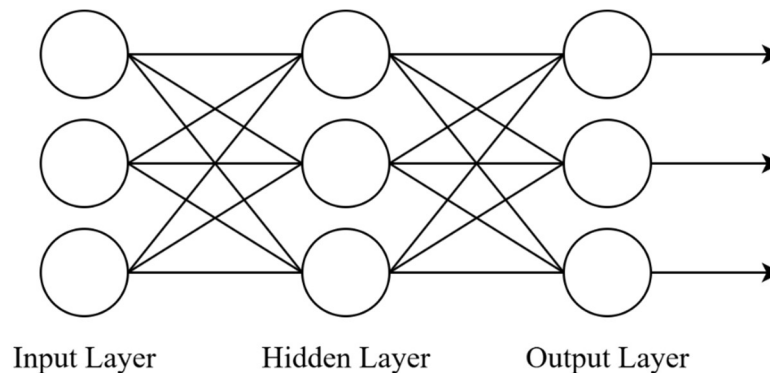


Figure 9: Multi-Layer Perceptron

The neurons of a multi-layer perceptron can be found in the hidden layers and output layers. The neurons execute the same tasks as the neuron in the perceptron. The neurons receive inputs, compute a weighted sum, add a bias term, and input the result into an activation function to produce an output.

Densely connected layers in a CNN are layers in which every neuron is connected to every neuron in the preceding and subsequent layers. The primary components of a densely connected layer are neurons, weights, biases, and an activation function. In a convolutional neural network, the flatten layer, densely connected layers, and output layer together form a multilayer

perceptron, where the densely connected layers serve as the hidden layers of the multi-layer perceptron. The weights and bias values of the multilayer perceptron component of the CNN are learned during training and are updated through backpropagation to minimize the loss function and improve the network's predictive accuracy.

2.4.7 Output Layer

The output layer provides the final predictions of the CNN. The neurons of the output layer execute the same tasks as the neurons in the densely connected layers. The outputs from the neurons are then passed to the softmax function, which converts them into probabilities between 0 and 1 for each class, such that the probabilities sum to 1. The number of neurons in the output layer corresponds to the number of classes. Let $k > 1$ be the number of classes, then the softmax function is defined as

$$\sigma: \mathbb{R}^k \rightarrow (0,1)^k$$

where $z = (z_1, z_2, \dots, z_k) \in \mathbb{R}^k$, $y = (y_1, y_2, \dots, y_k) \in (0,1)^k$, and

$$y_i = \frac{e^{z_i}}{\sum_{i=1}^k e^{z_i}}$$

The softmax function takes a k-tuple of real numbers as input and outputs a k-tuple of real numbers that lie in the interval (0,1). Each output component of the k-tuple of real numbers that lie in the interval (0,1) is computed by first applying the exponential function to the corresponding input component and then dividing by the sum of the exponentials of all input components. Since each component of the output, y_i , is normalized, the sum of all the components of the output will be 1. Thus, the output can be interpreted as a probability distribution over the k classes.

2.5 Multi-Stream Architectures

A multi-stream CNN is an architecture that consists of multiple feature-extraction branches, or streams, which process one or more inputs in parallel. The outputs of these streams can be combined in different ways depending on the selected fusion strategy. Multi-stream CNNs are used when inputs have different data representations, allowing the network to learn features from each representation before being combined to make predictions. The goal of providing multiple complementary sources of information is to achieve higher accuracy than what can be obtained using a single data representation alone. Simonyan and Zisserman proposed a multi-stream CNN for action recognition in video, consisting of a spatial stream and a temporal stream. The outputs of these streams were combined using two methods: averaging and a linear support vector machine (SVM) (2). Both multi-stream models outperformed the individual spatial and temporal streams, demonstrating the benefit of integrating complementary input representations (8). The model used in this study is a multi-stream CNN with two streams in which the outputs of these streams are concatenated and passed to a multilayer perceptron for classification. The specific details of the model can be found in Section 4.3 Model Selection.

3 Data

3.1 Collection

This study utilizes the KFall dataset, an open-access dataset developed by Xiaoqun Yu, Jaehyuk Jang, and Shuping Xiong at the Korea Advanced Institute of Science and Technology. The dataset was collected from 32 healthy young Korean male participants with no history of musculoskeletal disorder. Data acquisition was performed using an LPMS-B2 IMU configured at a frequency of 100 Hz. The LPMS-B2 contains a three-axis accelerometer, three-axis gyroscope, and three-axis magnetometer. The LPMS-B2 was placed on the lower back of each participant. Participants performed a variety of experimental activities, most of which were adopted from the SisFall dataset. These activities included routine movements, near-fall scenarios, and fall events categorized by preceding activities including walking, sitting, and standing. In addition, new static tasks and activities of daily living (ADLs) were introduced, while some existing actions were modified or removed. Static tasks were performed once, whereas other activities were repeated for five trials. Participants were instructed to carry out ADLs according to their daily habits, guided demonstrations were provided for fall scenarios when necessary. For safety, all fall activities were conducted on a mattress. Sensor data was transmitted through Bluetooth Dongle to a Raspberry Pi 4, which served as the host computer. Synchronized video was captured using a Raspberry Pi HQ camera, with synchronization between the sensor data and camera being handled by a multi-processing technique used in a custom Python-based program. The camera was positioned in the front side of the experimental area instead of directly ahead. After data collection, incomplete data was removed from the dataset (3-5).

In total there are 5,075 motion files. The dataset is organized into two main folders: `sensor_data` and `label_data`. The `sensor_data` folder contains 32 subfolders, each corresponding to an individual participant and labeled by subject ID. For example, the folder named SA06 represents data collected from subject 06. Each subfolder of the `sensor_data` folder contains the motion files corresponding to the associated participant. These files follow a standardized naming convention that encodes the subject ID, task ID, and trial ID. For example, the file S06T01R01, located in the SA06 subfolder, represents data from subject 06 (S06), task 01 (T01), and trial 01 (R01). In total, the dataset includes 34 distinct tasks. Each motion file contains numerical sensor data organized into 11 columns: `TimeStamp(s)`, `FrameCounter`, `AccX`, `AccY`, `AccZ`, `GyrX`, `GyrY`, `GyrZ`, `EulerX`, `EulerY`, and `EulerZ`. The `TimeStamp(s)` column records the elapsed time in seconds from the start of the trial, while the `FrameCounter` column contains the corresponding frame count. The `AccX`, `AccY`, and `AccZ` columns contain the acceleration due to gravity along the x, y, and z-axes, respectively. The `GyrX`, `GyrY`, and `GyrZ` columns contain the angular velocity in degrees per second along the x, y, and z-axes. Finally, the `EulerX`, `EulerY`, and `EulerZ` columns contain the Euler angle in degrees along the x, y, and z-axes.

The `label_data` folder contains 32 label files, each named according to the subject ID. For example, SA10_label.xlsx corresponds to subject 10. Each label file contains five columns: `Task Code (Task ID)`, `Description`, `Trial ID`, `Fall_onset_frame`, and `Fall_impact_frame`. The `Task Code (Task ID)` column specifies both the task code and its corresponding task ID. Only tasks with IDs ranging from 20 to 34 are included in these files, as they represent fall-related activities. The `Description` column provides a brief explanation of each task. The `Trial ID` column identifies the specific trial associated with the task. The `Fall_onset_frame` column contains the frame at which the subject begins to fall, while the `Fall_impact_frame` column contains the frame at which the

subject makes contact with the ground (“KFall: A Comprehensive Motion Dataset to Detect Pre-impact Fall for the Elderly based on Wearable Inertial Sensors”).

3.2 Exploration

The KFall dataset was explored to understand the distribution of fall and non-fall events across the dataset and across each participant. The sensor data was also investigated to gain an understanding of the sensor signals and their variability.

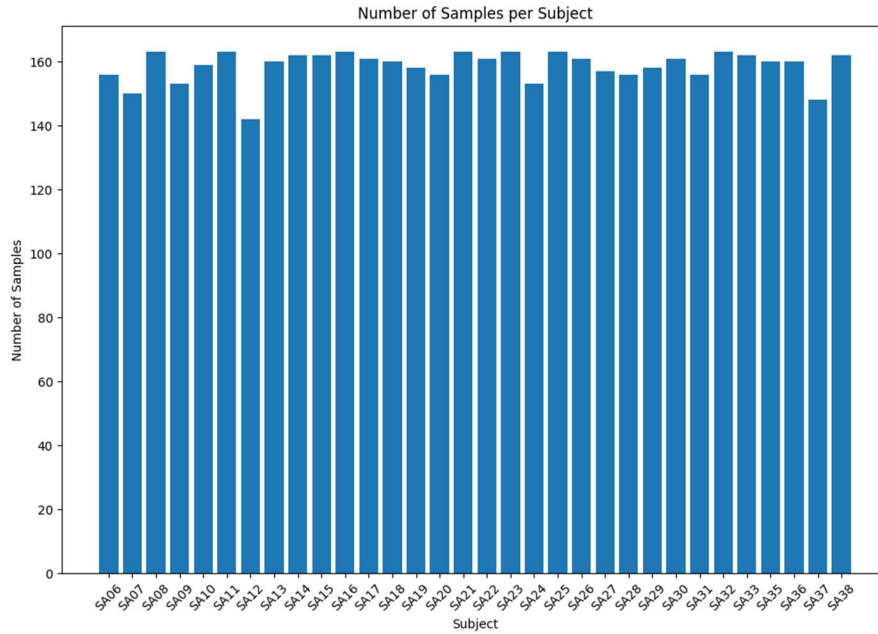


Figure 10: Number of Samples per Subject

The total number of samples per subject was examined to understand the data distribution per participant. This distribution can be seen in Figure 10. Overall, the dataset is relatively evenly distributed across participants, suggesting good diversity.

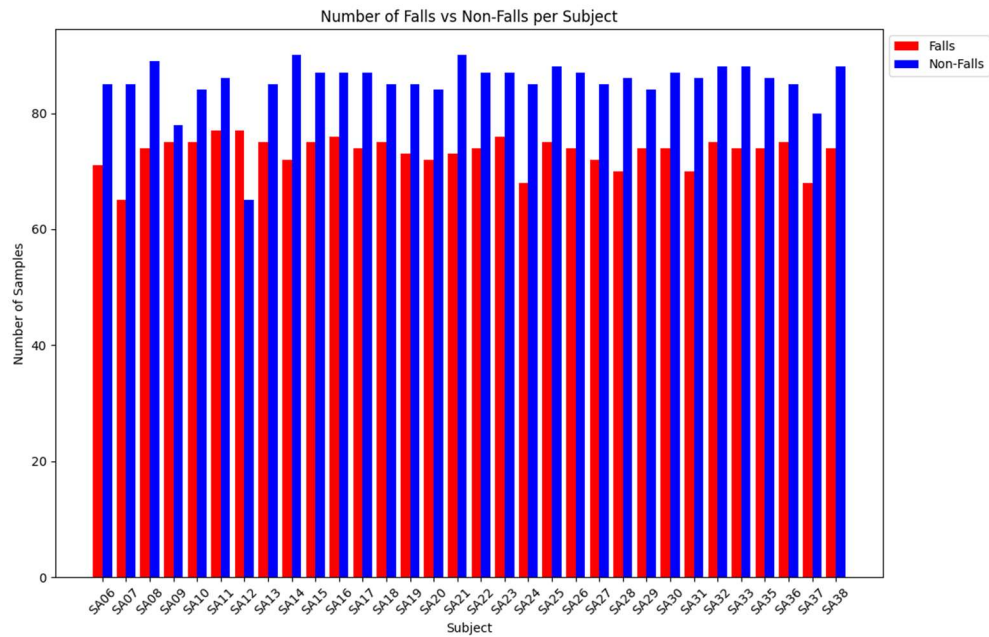


Figure 11: Number of Fall and Non-Falls per Subject

The distribution of fall and non-fall events per subject was also examined. This distribution can be seen in Figure 11. There is some variation in the distribution, but overall, there seems to be good distribution among the different subjects.

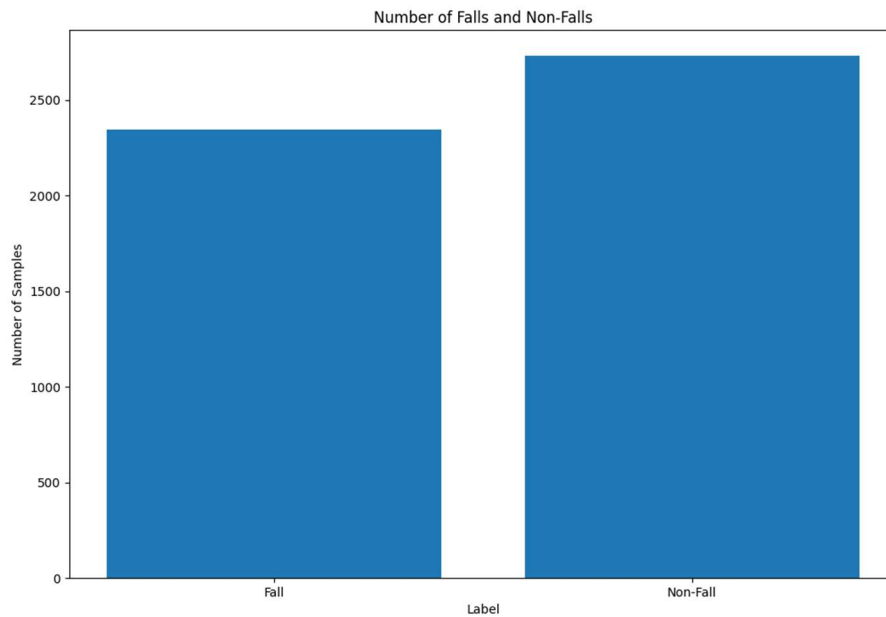


Figure 12: Number of Fall and Non-Falls

The number of fall and non-fall events was examined to understand the class distribution of the dataset. This distribution can be seen in Figure 12. There is a total of 2346 fall events and 2729

non-fall events in the dataset, indicating a mild class imbalance. An imbalanced class distribution leads to a model biasing the class with more samples. To get a clear picture of how the model in this study is performing metrics such as precision, recall, and F1-score will be used.

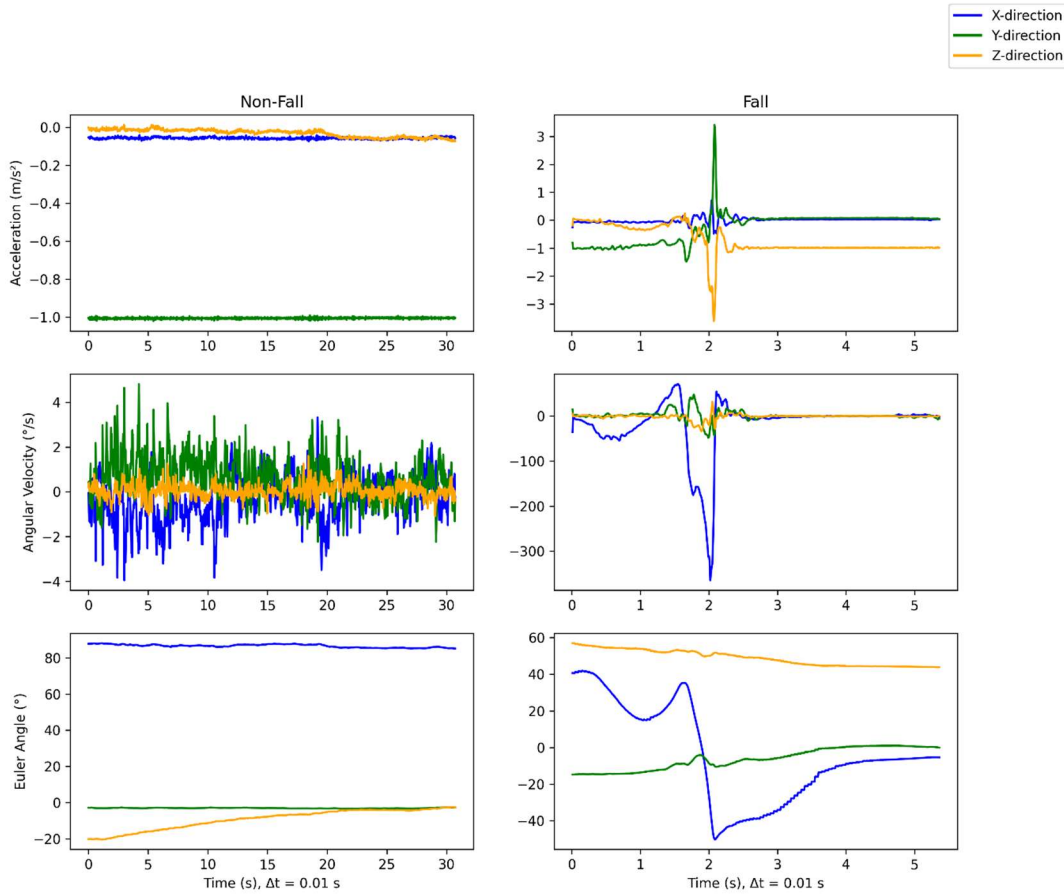


Figure 13: Subject 06 Fall and Non-Fall Sensor Signal Plots

In Figure 13, the sensor data for S06T01R01 is plotted in the first column and the sensor data for S06T20R01 is plotted in the second column. S06T01R01 is a motion file that does not contain a fall whereas S06T20R01 is a motion file that does contain a fall. The first row contains the plots of acceleration in meters per second squared along the x, y, and z-axes for both samples. The second row contains the plots of angular velocity in degrees per second along the x, y, and z-axes for both samples. The third row contains the plots of Euler angle in degrees along the x, y, and z-axes for both samples. Notable differences can be observed between the two types of events. Fall events exhibit sudden and pronounced changes in both acceleration and angular velocity, reflecting the rapid motion and impact associated with a fall. In contrast, non-fall events show smoother and more gradual variations over time. These distinctions suggest that temporal patterns in acceleration and angular velocity can be effectively leveraged by a machine learning model to differentiate between fall and non-fall events.

4 Methodology

4.1 Data Preprocessing

The raw sensor data was preprocessed prior to feature selection and model training. The initial preprocessing step involved generating appropriate labels for the data and improving the folder structure of the dataset. To achieve this, a new column named “Label” was added to each motion file. This column assigns a binary value of 0 or 1 to every timestamp. For motion files without a fall event, all entries in the Label column were set to 0. For motion files containing a fall, entries were assigned a value of 1 for timestamps occurring between the fall onset frame and fall impact frame, while all other timestamps were assigned a value of 0. This approach ensures that each timestamp within a motion sequence is explicitly labeled. After incorporating labels directly into the motion files, the separate label_data folder and its associated label files were no longer necessary and were removed. Consequently, the dataset was reorganized such that the labeled data consists of 32 subfolders, each corresponding to an individual participant and identified by subject ID. Each subfolder contains the motion files for the respective participant.

The final preprocessing step involved segmenting the time-series data into one-second windows. For each window, values were extracted from all six sensor channels, including acceleration and angular velocity along the x, y, and z-axes. Euler angle data was excluded to avoid potential data leakage. If a window contained fewer values than required to represent a full one-second interval, zero-padding was applied to maintain a consistent window length. For each sensor channel, the windowed segments were stored in separate lists, one for each sensor channel, i.e., acceleration along the x-axis, acceleration along the y-axis, acceleration along the z-axis, angular velocity along the x-axis, angular velocity along the y-axis, and angular velocity along the z-axis. These six channel-specific lists were then stored into a list, where corresponding indices across the channel-specific lists represent the same one-second time window. This list was subsequently converted into a multi-dimensional NumPy array that had a shape of (6, 42465, 100), where the six channels correspond to acceleration and angular velocity along the x, y, and z axes, 42,465 represents the number of one-second windows, and 100 represents the number of time samples per window. This NumPy array was saved in a NumPy file named windowed_data. Each one-second window was assigned a single label based on the original Label column. A window was labeled as a fall with a value of 1 if at least one timestamp within that window had a label of 1 in the Label column. Otherwise, the window was labeled as a non-fall with a value of 0. Thus, each set of six synchronized sensor windows corresponds to one label. The labels were saved in a NumPy array with shape (42465,). This NumPy array was saved in a NumPy file named labels.

4.2 Feature Extraction

The feature extraction process was applied to the data array of shape (6, 42465, 100). The CWT was computed independently for each window and sensor channel. Specifically, for every window, the CWT was applied independently to all six sensor channels: acceleration along the x-axis, acceleration along the y-axis, acceleration along the z-axis, angular velocity along the x-axis, angular velocity along the y-axis, and angular velocity along the z-axis. A sampling frequency of 100 Hz (sampling period of 0.01 s) was used, along with 50 scales and the Morlet wavelet. Following the transformations, the magnitudes of the wavelet coefficients were computed to obtain a time-frequency representation for each channel. For each window, the coefficients were grouped by sensor modality. The three acceleration channels were stacked along a new axis to form a three-channel array representing acceleration, while the three

channels for angular velocity were similarly stacked to form a three-channel array representing angular velocity. These acceleration and angular velocity three-channel arrays were stored in separate lists. The separate lists were then stored in a list that was converted to a multi-dimensional NumPy array with shape (2, 42465, 3, 50, 100). The first dimension corresponds to the 2 sensor modalities: acceleration and gyroscope, the second to the number of windows, the third to the number of channels per modality, the fourth to the number of scales, and the fifth to the number of time samples. This NumPy array was saved in a NumPy file named scalogram_data. Finally, no modifications to the labels NumPy file were needed because the labels generated during the windowing process were aligned with the extracted features, such that each label corresponds to a pair of three-channel arrays, one for acceleration and one for gyroscope, for a given window.

4.3 Model Selection

The model architecture was implemented using TensorFlow and Keras. Since each sample consists of two three-channel representations, a multi-stream CNN was used. One stream processes the three-channel arrays that correspond to the acceleration modality, while the second stream processes the three-channel arrays that correspond to the gyroscope modality. The input shape for both streams is (50,100,3). The model consists of two parallel convolutional streams, one for each sensor modality. Each stream contains one convolutional block, where each block includes a 2D convolutional layer followed by a max-pooling layer. The convolutional layers use 32 filters with a 3x3 kernel, a stride of 1, no padding, and the ReLU activation function. The max-pooling layers use a 2x2 window with a stride of 1. The resulting feature maps are flattened and the outputs from the two streams are concatenated. The merged features are passed through a fully connected layer with 128 neurons and the ReLU activation function. Finally, the output layer contains a single neuron with a sigmoid activation function, producing a probability score for binary classification.

Layer (type)	Output Shape	Param #	Connected to
input_1 (InputLayer)	[(None, 50, 100, 3)]	0	[]
input_2 (InputLayer)	[(None, 50, 100, 3)]	0	[]
conv2d (Conv2D)	(None, 50, 100, 32)	896	['input_1[0][0]']
conv2d_1 (Conv2D)	(None, 50, 100, 32)	896	['input_2[0][0]']
activation (Activation)	(None, 50, 100, 32)	0	['conv2d[0][0]']
activation_1 (Activation)	(None, 50, 100, 32)	0	['conv2d_1[0][0]']
max_pooling2d (MaxPooling2D)	(None, 25, 50, 32)	0	['activation[0][0]']
max_pooling2d_1 (MaxPooling2D)	(None, 25, 50, 32)	0	['activation_1[0][0]']
flatten (Flatten)	(None, 40000)	0	['max_pooling2d[0][0]']
flatten_1 (Flatten)	(None, 40000)	0	['max_pooling2d_1[0][0]']
concatenate (Concatenate)	(None, 80000)	0	['flatten[0][0]', 'flatten_1[0][0]']
dense (Dense)	(None, 128)	10240128	['concatenate[0][0]']
dense_1 (Dense)	(None, 1)	129	['dense[0][0]']
=====			
Total params: 10,242,049			
Trainable params: 10,242,049			
Non-trainable params: 0			

Figure 14: Base Model Architecture

The model summary of this model architecture can be seen in Figure 14. This model architecture will be optimized and tuned, which will result in a different architecture with different hyperparameters. Further details are provided in Section 5 Results.

4.4 Model Training

The scalogram_data and labels NumPy files were first loaded in. The scalogram_data NumPy file contained the samples while the labels NumPy file contained the corresponding labels. The NumPy array from the scalogram_data file initially had a shape of (2, 42465, 3, 50, 100) and the labels file had a shape of (42465,). The NumPy array from the scalogram_data file was transposed to correctly structure the samples, sensor modalities, and image dimensions for model input. The resulting shape was (42465, 2, 50, 100, 3). The dataset was divided into a training set and a temporary holdout set. Stratified sampling was applied using the class labels to ensure that the class distribution was preserved across splits, with 70% of the data assigned to training and 30% assigned to the temporary set. The data was shuffled prior to splitting, and a fixed random seed was used to ensure reproducibility of the results. The temporary set was then further split into validation and test sets. This second split was performed with equal proportions, resulting in 15% of the original dataset for validation and 15% for testing. Stratified sampling was again used to maintain consistent class distributions in both subsets, while shuffling and the same random seed ensured reproducibility. Each sample in the dataset consists of two sensor modalities: acceleration and gyroscope data. After splitting the dataset into training, validation, and test sets, the two modalities were separated into independent inputs by indexing the second dimension of the array (i.e., selecting channel 0 for acceleration and channel 1 for gyroscope). To ensure stable training and consistent feature scaling, z-score normalization was applied separately to each modality. The mean and standard deviation were computed using only the training set for both acceleration and gyroscope data. This prevents data leakage by ensuring that information from the validation and test sets does not influence the normalization parameters. The computed training statistics were then used to normalize the training, validation, and test sets for each modality. A small constant, 1×10^{-8} , was added to the standard deviation during normalization to prevent numerical instability from division by zero. The initial model architecture was explained in Section 4.3 Model Selection. The model was built and trained using Tensorflow and Keras. After defining the architecture, the model was compiled. The Adam optimizer with an initial learning rate of 0.001 was used for parameter updates. A learning rate scheduler was used to adjust the learning rate during training. The learning rate scheduler monitored validation loss and if validation loss did not improve within 3 epochs the learning rate was reduced by 50%. The minimum learning rate was set as 0.000001. The learning rate was tracked at the end of each epoch using a custom callback. The binary cross-entropy loss function was used due to the classification task being a binary classification problem, and accuracy was the evaluation metric used to monitor the model's performance during training. The normalized acceleration and gyroscope training data were provided as separate input streams, along with their corresponding labels for training. Validation data for both modalities was included to evaluate model performance after each epoch. Training was conducted for 50 epochs using a batch size of 32. After training, the model was saved as a Keras file so it could be reloaded later for evaluation on the testing set, which will be discussed further in Section 4.5 Model Evaluation. The Matplotlib library was used to create plots showing the model's performance over time. One plot showed training accuracy, validation accuracy, and scaled learning rate across epochs. The learning rates

were scaled to fit within the same range as the accuracy values. The learning rates were multiplied by a scaling factor that was equal to the quotient of the maximum training accuracy and maximum learning rate. The second plot showed training loss, validation loss, and scaled learning rate across epochs. The learning rates were multiplied by a scaling factor that was equal to the quotient of the maximum training loss and maximum learning rate. The output of training script along with the training progress was saved to a text file.

4.5 Model Evaluation

The test data used for evaluation was obtained using the same preprocessing pipeline described in Section 4.4 Model Training. The NumPy arrays were loaded and the array containing the samples was transposed to a shape of (42465, 2, 50, 100, 3). The data was split into a 70% training set and a 30% temporary set, with stratified sampling applied to preserve class distributions. The temporary set was split into a 15% validation set and 15% testing set, again using stratified sampling. In both splitting steps, shuffling with the same fixed random seed as the one used during model training ensured that the exact same test set was reproduced for model evaluation. The sensor modalities were then separated by indexing the second dimension of the array. Finally, z-score normalization was applied separately to each modality. After obtaining the test sets for each modality, each trained model was loaded, and predictions were generated on the normalized test set for both sensor modalities. A precision–recall curve was generated for all models. The precision-recall curves were plotted on the same figure and distinguished using a legend, and the resulting plot was saved. In addition, a confusion matrix was produced and saved for each model to visualize classification performance. Finally, a classification report was generated and saved for each model, summarizing precision, recall, F1-score, accuracy, macro average, and weighted average.

5 Results

Seven multi-stream CNN models were trained and evaluated. The first model that was trained and evaluated was the model discussed in Section 4.3 Model Selection.

Experiment ID	Configuration	Trainable Parameters	Non-Trainable Parameters	Total Parameters	Training Accuracy	Training Loss	Validation Accuracy	Validation Loss
001	Base Model	10,242,049	0	10,242,049	0.9984	0.0053	0.9765	0.1247
002	Replaced max pooling and flatten layers with global average pooling layers.	10,241	0	10,241	0.9733	0.0766	0.9699	0.0924
003	Added one convolutional block per stream, 32 filters each	28,737	0	28,737	0.9795	0.0579	0.9749	0.0797
004	Added convolutional block to each stream	47,233	0	47,233	0.9889	0.0306	0.9766	0.0727
005	Removed one convolutional block from each stream and updated filters to (32, 64)	55,425	0	55,425	0.9856	0.0407	0.9755	0.0778
006	Reduced filters to (32, 32) and added dropout to fully connected layers	28,737	0	28,737	0.9806	0.0549	0.9755	0.0784
007	Added batch normalization	28,993	256	29,249	0.9756	0.0708	0.9728	0.0841

Figure 15: Experiments Table

Figure 15 is a table that compares the performance of the models during training. The models are organized by Experiment ID and the changes made from model to model are stated in the Configuration column. The table shows the trainable parameters, non-trainable parameters, total parameters, training accuracy, training loss, validation accuracy, and validation loss for each of the model configurations used during training. Model 004 achieved the lowest validation loss. Model 003, 005, and 006 also achieved a validation loss within 0.01 of the validation loss achieved by Model 004. Model 004 also achieved the best validation accuracy. All other models achieved a validation accuracy within 0.01 of the validation accuracy achieved by Model 004. Model 001 achieved the highest training accuracy with Model 004 achieving the second highest. The difference between Model 001's training accuracy and Model 004's training accuracy was 0.0095, however Model 004 has 10,194,816 less parameters. Model 001 achieved the lowest training loss with Model 004 achieving the second lowest training loss. Model 004 seems to have performed well during training, achieving the highest validation accuracy and lowest validation loss while also attaining the second-best training accuracy and training loss, despite having 10,194,816 fewer parameters than Model 001.

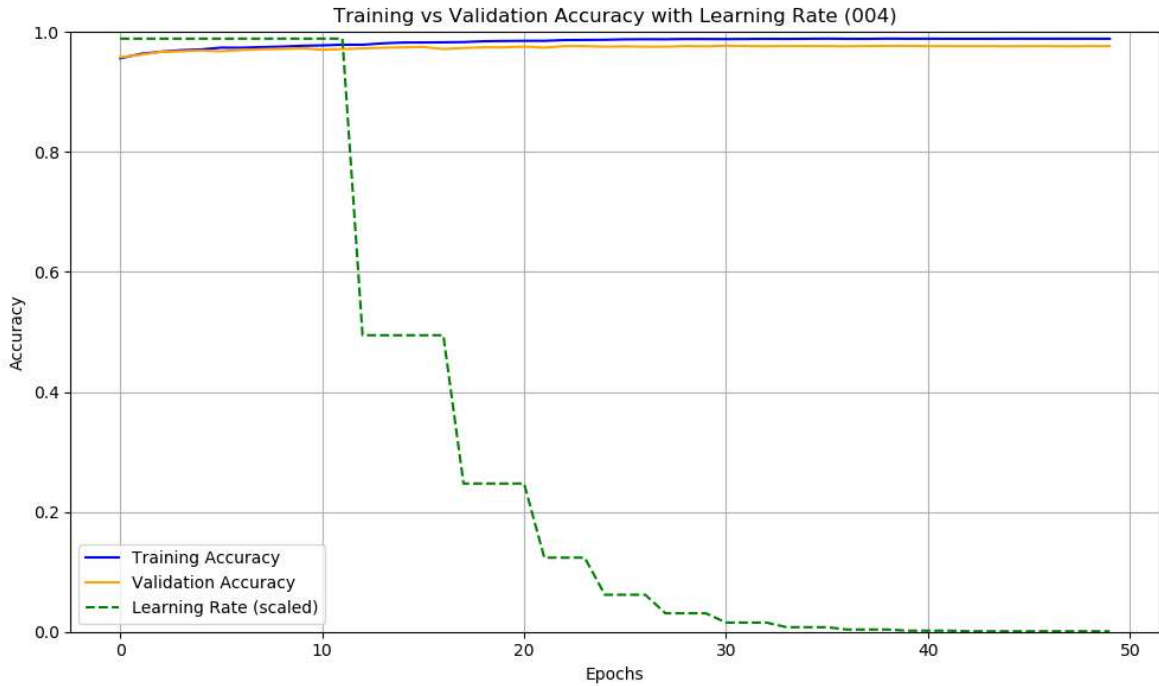


Figure 16: Model 004 Training Accuracy, Validation Accuracy, Learning Rate (scaled)
 Figure 16 is the plot of training accuracy, validation accuracy, and the scaled learning rate for Model 004 across 50 epochs. The training and validation accuracies are initially high and slightly increase until they both converge around epoch 30. The gap between training and validation accuracy is small throughout all 50 epochs. The learning rate decreases throughout training and eventually reaches the learning rate threshold around epoch 45.

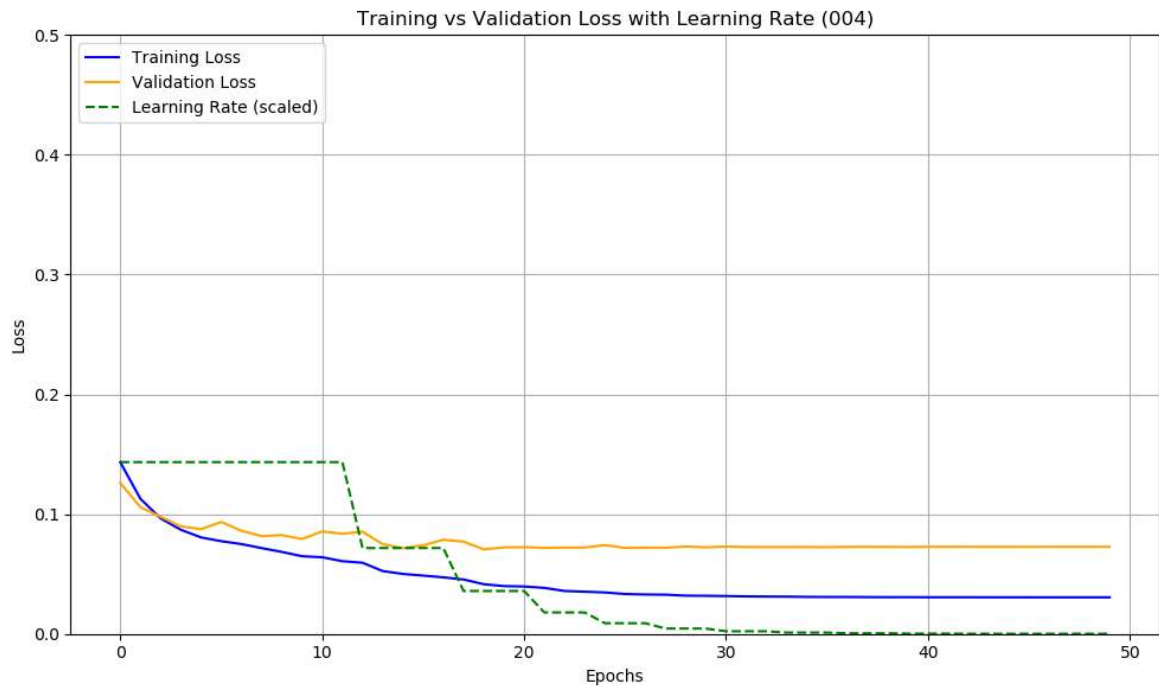


Figure 17: Model 004 Training Loss, Validation Loss, Learning Rate (scaled)

Figure 17 is the plot of training loss, validation loss, and the scaled learning rate for Model 004 across 50 epochs. The training loss steadily decreases and converges after epoch 30. The validation loss decreases with some fluctuations and converges after epoch 30. There is a small gap between the training and validation loss, however, both curves are under 0.1 loss. As also seen in Figure 16, the learning rate decreases and eventually reaches the learning rate threshold around epoch 45.

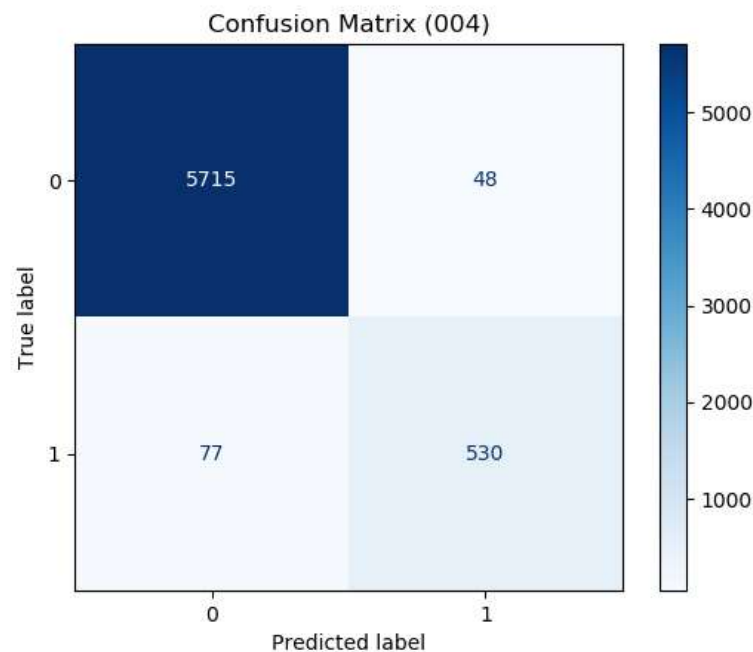


Figure 18: Model 004 Confusion Matrix

Figure 18 is the confusion matrix for Model 004. The confusion matrix shows 5715 true negatives, 530 true positives, 48 false negatives, and 77 false positives.

Classification Report (004)				
-	Precision	Recall	F1-Score	Support
0	0.99	0.99	0.99	5,763
1	0.92	0.87	0.89	607
Accuracy	-	-	0.98	6,370
Macro Average	0.95	0.93	0.94	6,370
Weighted Average	0.98	0.98	0.98	6,370

Figure 19: Model 004 Classification Report

Figure 19 is the classification report for Model 004. The classification report shows that the model had an accuracy of 98% on the testing data. There was a total of 5,763 samples that were non-falls in the testing data. For non-falls the precision was 99%, the recall was 99%, and the F1-score was 99%. There was a total of 607 samples that were falls in the testing data. For falls the precision was 92%, the recall was 87%, and the F1-score was 89%. The macro averages for precision, recall, and F1-score were 95%, 93%, and 94%, respectively. The weighted averages for precision, recall, and F1-score were 98%, 98%, and 98%, respectively.

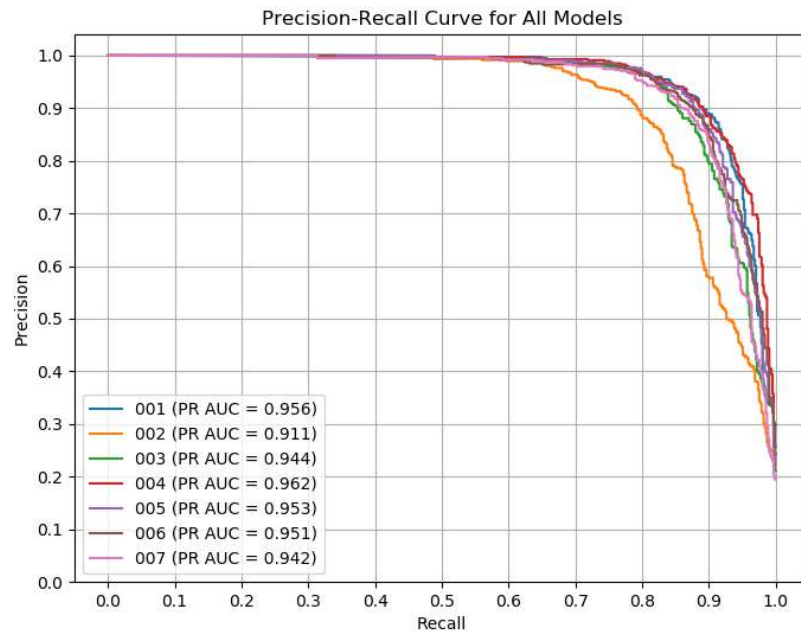


Figure 20: Precision-Recall Curves for All Models

Figure 20 shows the precision-recall curves for all the models along with their area under the curve (AUC) values. Model 004 had the highest AUC value with Model 001 having the second highest AUC value. All the models achieved an AUC value of at least 90%.

6 Discussion

All models achieved AUC values of at least 0.90, indicating strong performance in distinguishing between fall and non-fall events. This suggests that the multi-stream CNNs maintain a good balance between correctly identifying falls and minimizing false positives across different decision thresholds. Model 004 demonstrated the strongest overall performance among all evaluated models. It achieved the highest validation accuracy and lowest validation loss, while also recording the second-best training accuracy and training loss. In addition, Model 004 produced the highest precision–recall AUC value of 0.962, indicating strong discrimination between fall and non-fall events across classification thresholds. Model 004 contained three convolutional blocks in each stream, the greatest depth among the tested architectures. This suggests that the additional convolutional layers may have improved the model’s ability to learn features from the accelerometer and gyroscope inputs. Model 001 achieved the second-highest AUC value of 0.956. It also obtained the best training accuracy and lowest training loss, along with the second-highest validation accuracy. However, Model 001 showed evidence of overfitting, as indicated by the gap in training loss, 0.0053, and validation loss, 0.1247. This is likely due to its large number of trainable parameters and the absence of regularization techniques. While Model 001 exhibited overfitting, it still performed competitively across training and validation metrics compared with models that had fewer parameters and incorporated regularization techniques to reduce overfitting. This suggests that the model selection of a multi-stream CNN was a good choice for learning the CWT representations of the acceleration and gyroscope sensor modalities. Another important thing to note is that Model 004 used 10,194,816 fewer parameters than Model 001 while outperforming it on validation metrics. This suggests that highly complex multi-stream CNN architectures may not be necessary for effective fall detection.

7 Conclusion

This study developed a machine learning pipeline for fall detection using the KFall dataset. Time-series signals from the accelerometer and gyroscope modalities were windowed and transformed into time-frequency representations using the CWT. Seven multi-stream CNNs were trained and tested with all models achieving AUC values of at least 0.90, indicating that the models could distinguish between fall and non-fall events. Model 004 performed well, achieving the highest validation accuracy, 0.9766, lowest validation loss, 0.0727, and highest AUC value, 0.962. The results demonstrate that multi-stream CNNs are effective at distinguishing falls and non-falls when given time-frequency representations of acceleration and gyroscope signals obtained from the CWT. In addition, highly complex multi-stream architectures with millions of parameters may not be necessary for fall detection. Future work may involve training and evaluating additional architectures with alternative hyperparameter settings, implementing loss functions designed for class imbalance, comparing multi-stream and single-modality CNN approaches, and optimizing the final model through quantization and pruning for efficient deployment on embedded hardware.

Bibliography

- Centers for Disease Control and Prevention. "Older Adult Falls Data." *Older Adult Fall Prevention*, 26 Feb. 2026, www.cdc.gov/falls/data-research/index.html.
- "Inertial Measurement Unit (IMU) – An Introduction." *Advanced Navigation*, 13 Feb. 2023, www.advancednavigation.com/tech-articles/inertial-measurement-unit-imu-an-introduction/.
- Kerns, Nancy. "The Dogs We Love: Too Little Time Together." *Whole Dog Journal*, 10 July 2018, www.whole-dog-journal.com/blog/the-dogs-we-love-too-little-time-together/.
- "KFall: A Comprehensive Motion Dataset to Detect Pre-impact Fall for the Elderly based on Wearable Inertial Sensors." sites.google.com/view/kfalldataset.
- Kromydas, Bill. "Convolutional Neural Network (CNN): A Complete Guide." *LearnOpenCV by BIGVISION*, 18 Jan. 2023, learnopencv.com/understanding-convolutional-neural-networks-cnn/.
- National Institute on Aging. "Falls and Fractures in Older Adults: Causes and Prevention." 12 Sept. 2022, www.nia.nih.gov/health/falls-and-falls-prevention/falls-and-fractures-older-adults-causes-and-prevention.
- Simonyan, Karen, and Zisserman, Andrew. "Two-Stream Convolutional Networks for Action Recognition in Videos." 2014, pp. 1-11. arxiv.org/pdf/1406.2199.
- Yu, Xiaoqun, et al. "A Large-Scale Open Motion Dataset (KFall) and Benchmark Algorithms for Detecting Pre-impact Fall of the Elderly Using Wearable Inertial Sensors." *Frontiers in Aging Neuroscience*, vol. 13, 2021, pp. 1-14. doi.org/10.3389/fnagi.2021.692865.

Appendix

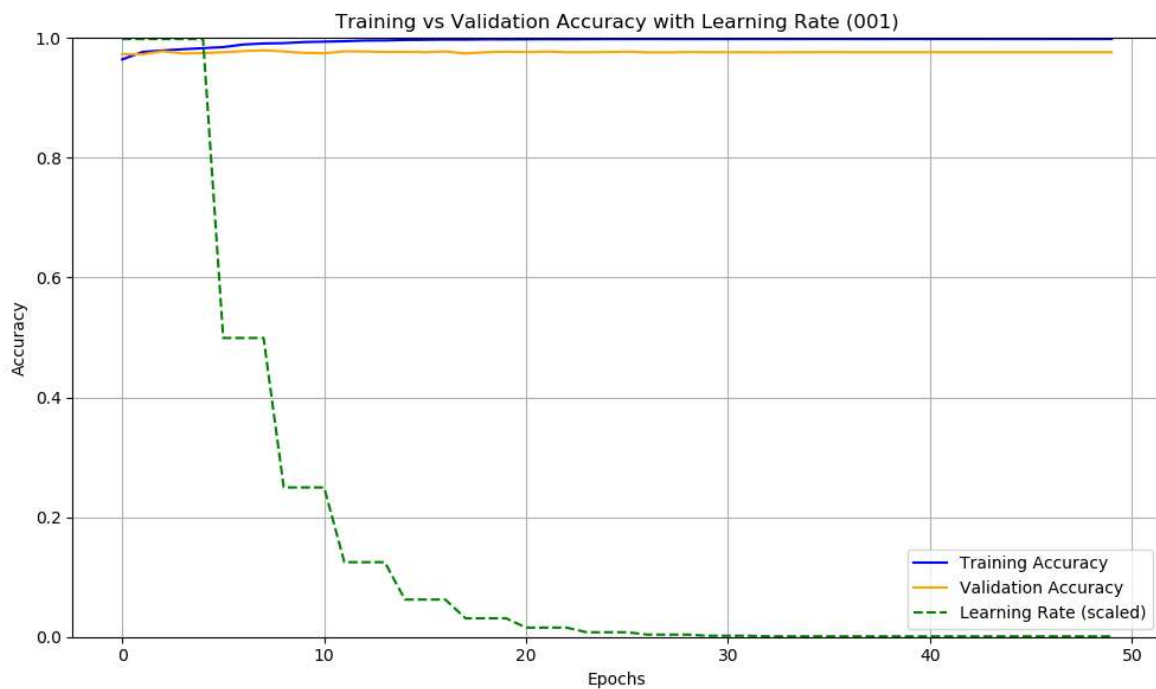


Figure 21: Model 001 Training Accuracy, Validation Accuracy, Learning Rate (scaled)

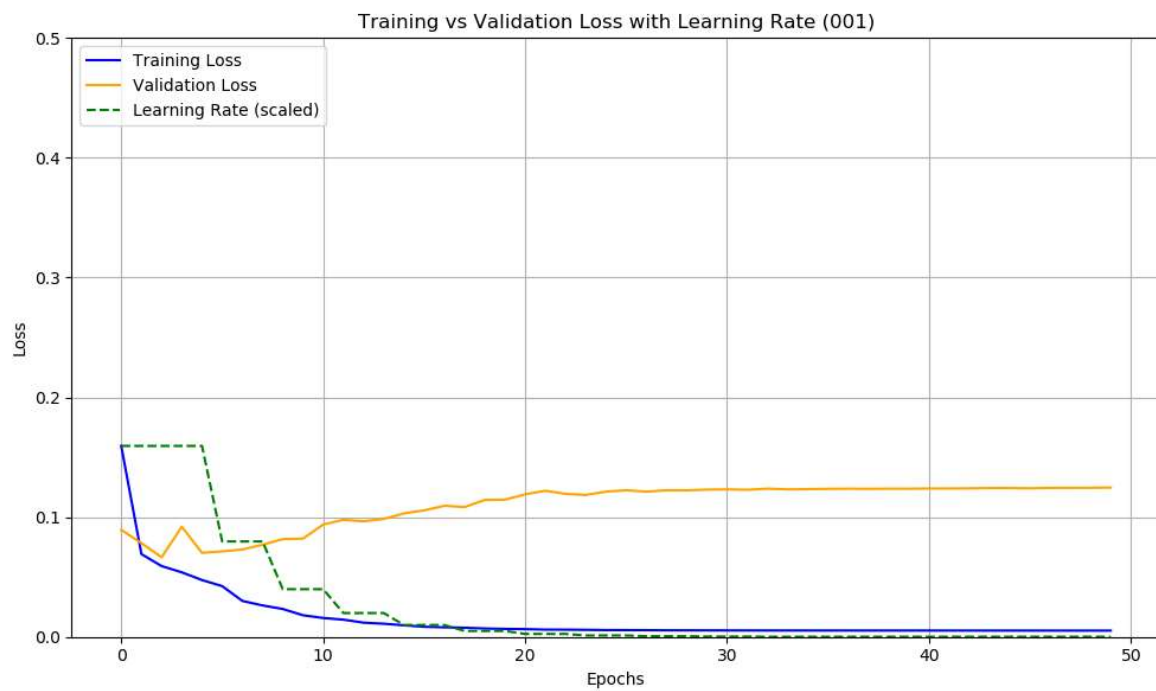


Figure 22: Model 001 Training Loss, Validation Loss, Learning Rate (scaled)

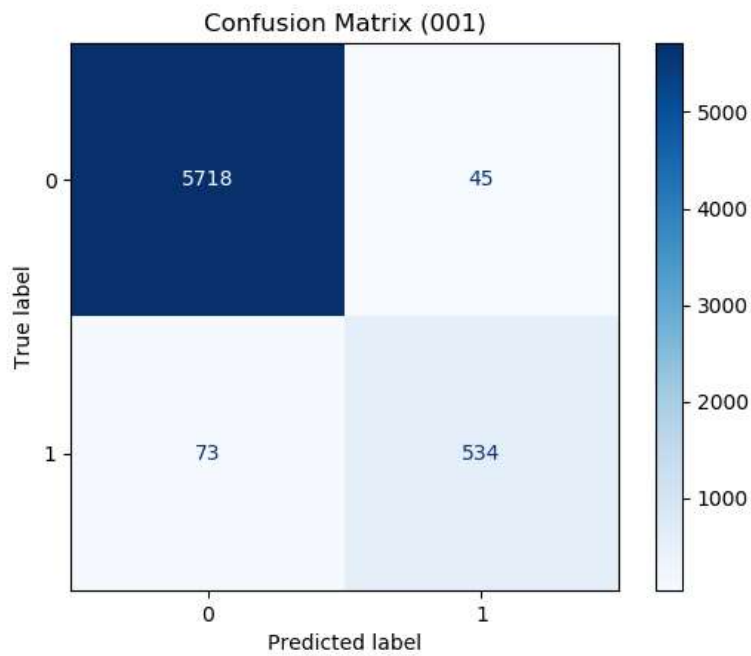


Figure 23: Model 001 Confusion Matrix

Classification Report (001)				
-	Precision	Recall	F1-Score	Support
0	0.99	0.99	0.99	5,763
1	0.92	0.88	0.90	607
Accuracy	-	-	0.98	6,370
Macro Average	0.95	0.94	0.95	6,370
Weighted Average	0.98	0.98	0.98	6,370

Figure 24: Model 001 Classification Report

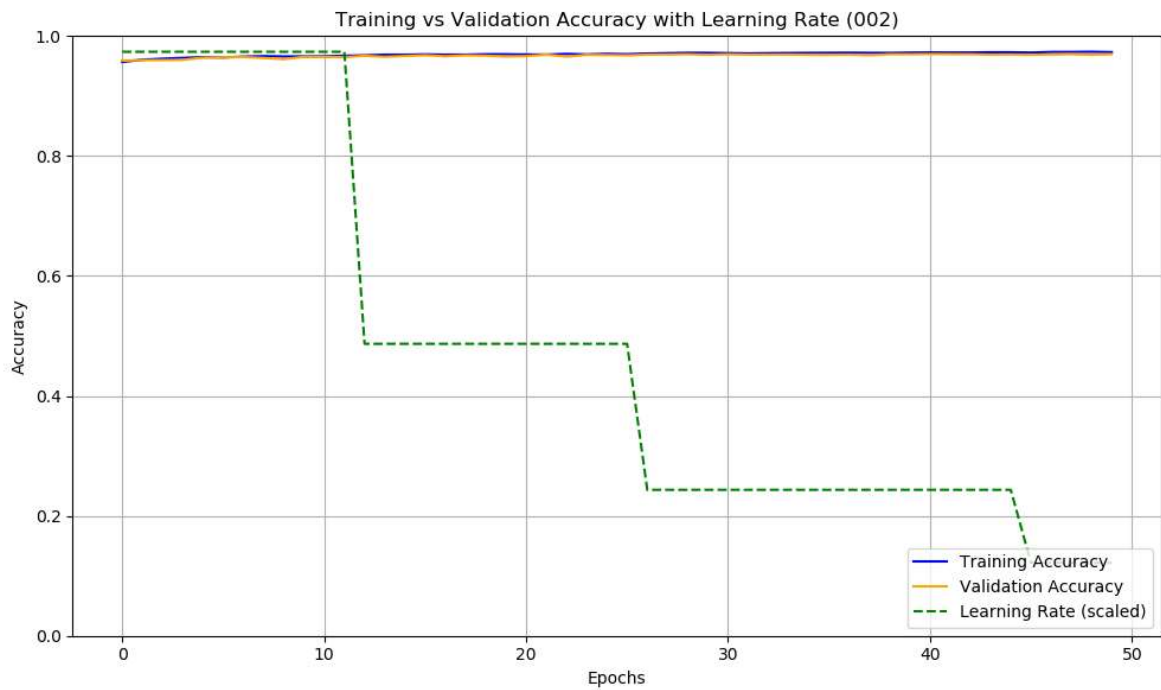


Figure 25: Model 002 Training Accuracy, Validation Accuracy, Learning Rate (scaled)

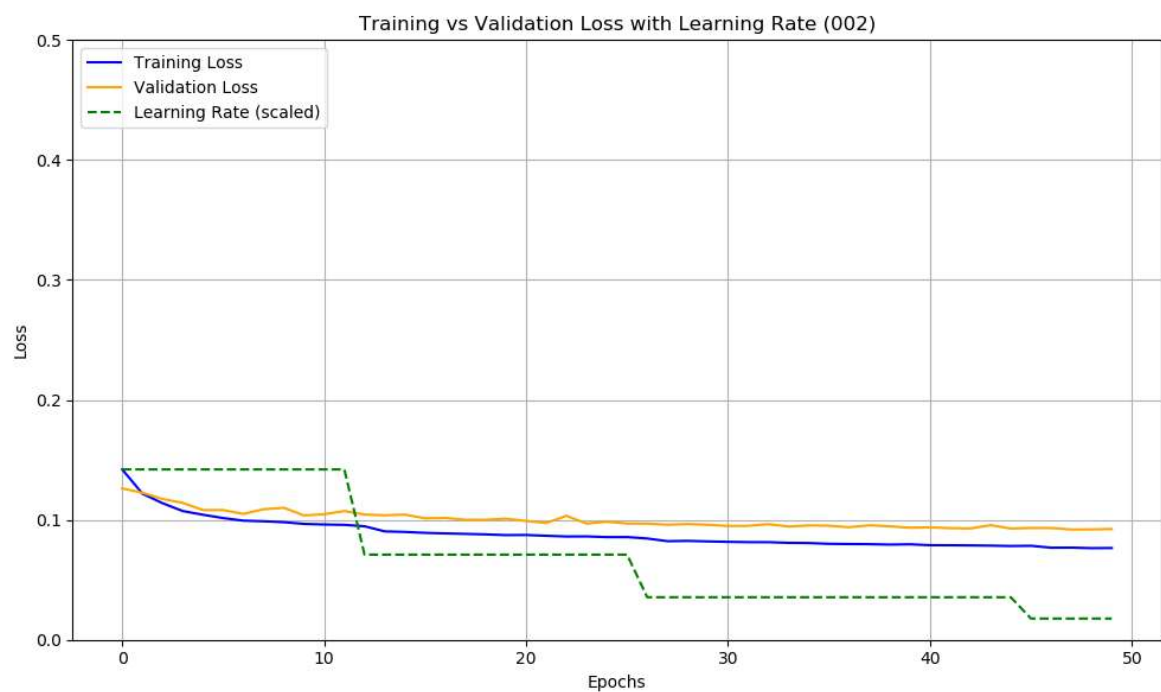


Figure 26: Model 002 Training Loss, Validation Loss, Learning Rate (scaled)

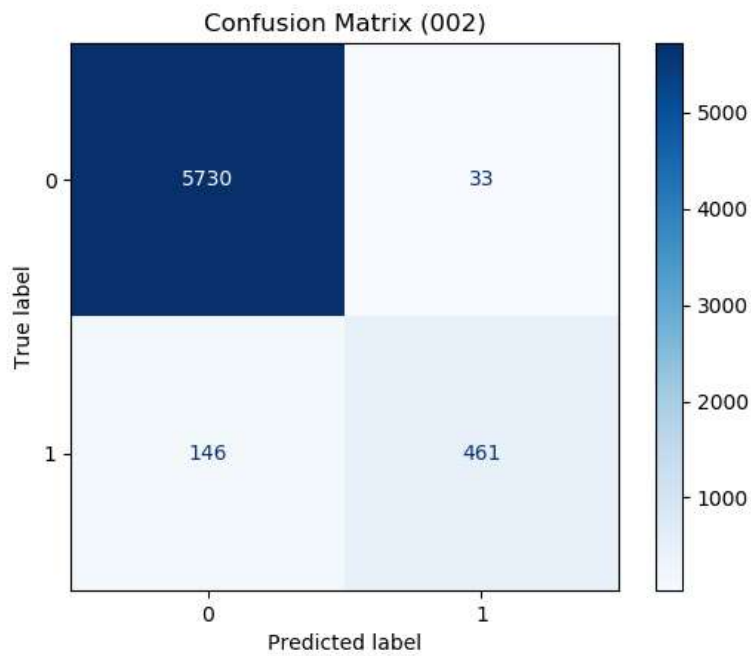


Figure 27: Model 002 Confusion Matrix

Classification Report (002)				
-	Precision	Recall	F1-Score	Support
0	0.98	0.99	0.98	5,763
1	0.93	0.76	0.84	607
Accuracy	-	-	0.97	6,370
Macro Average	0.95	0.88	0.91	6,370
Weighted Average	0.97	0.97	0.97	6,370

Figure 28: Model 002 Classification Report

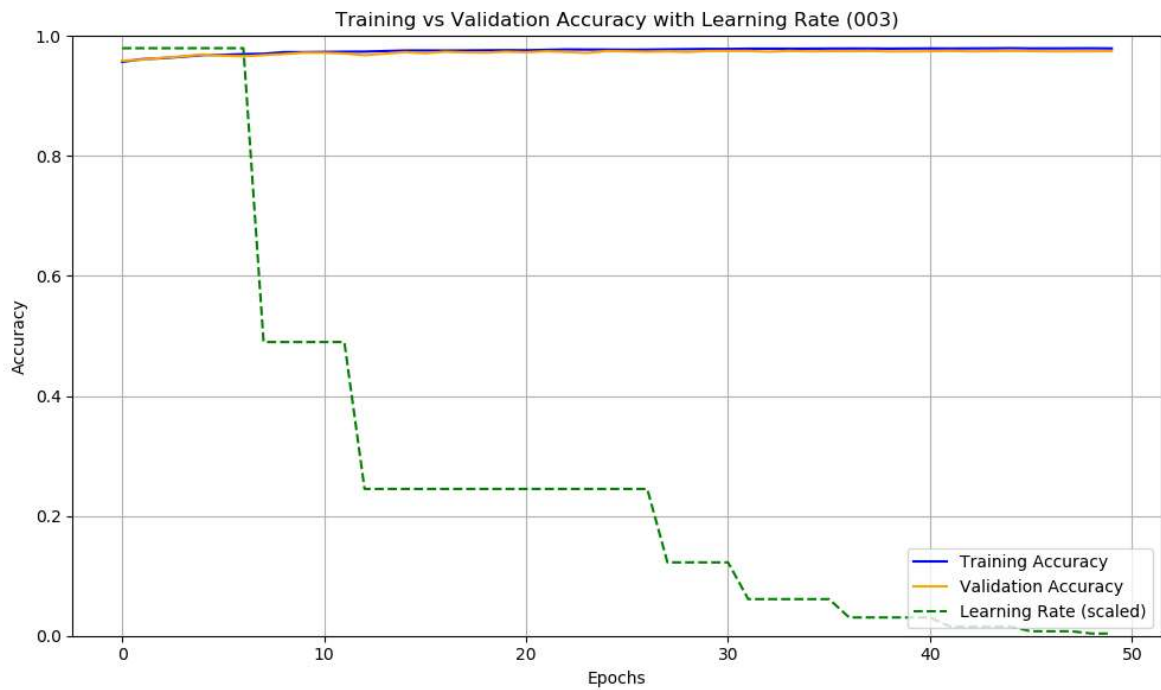


Figure 29: Model 003 Training Accuracy, Validation Accuracy, Learning Rate (scaled)

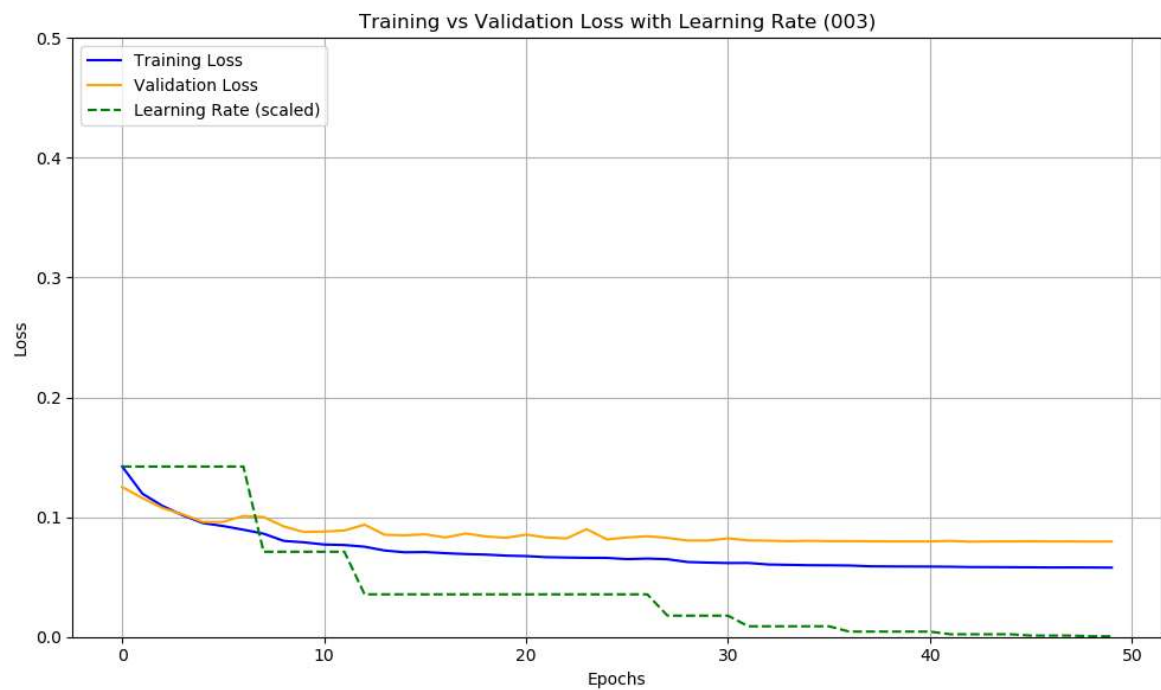


Figure 30: Model 003 Training Loss, Validation Loss, Learning Rate (scaled)

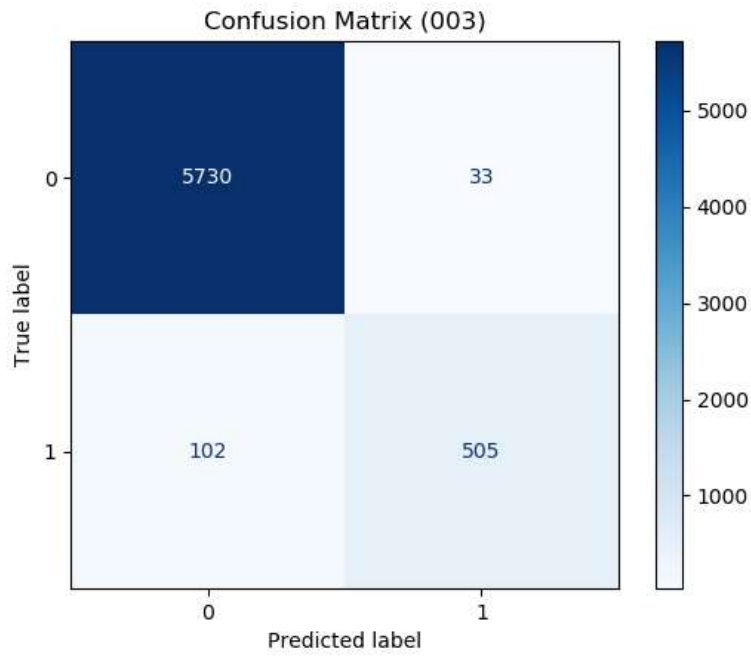


Figure 31: Model 003 Confusion Matrix

Classification Report (003)				
-	Precision	Recall	F1-Score	Support
0	0.98	0.99	0.99	5,763
1	0.94	0.83	0.88	607
Accuracy	-	-	0.98	6,370
Macro Average	0.96	0.91	0.94	6,370
Weighted Average	0.98	0.98	0.98	6,370

Figure 32: Model 003 Classification Report

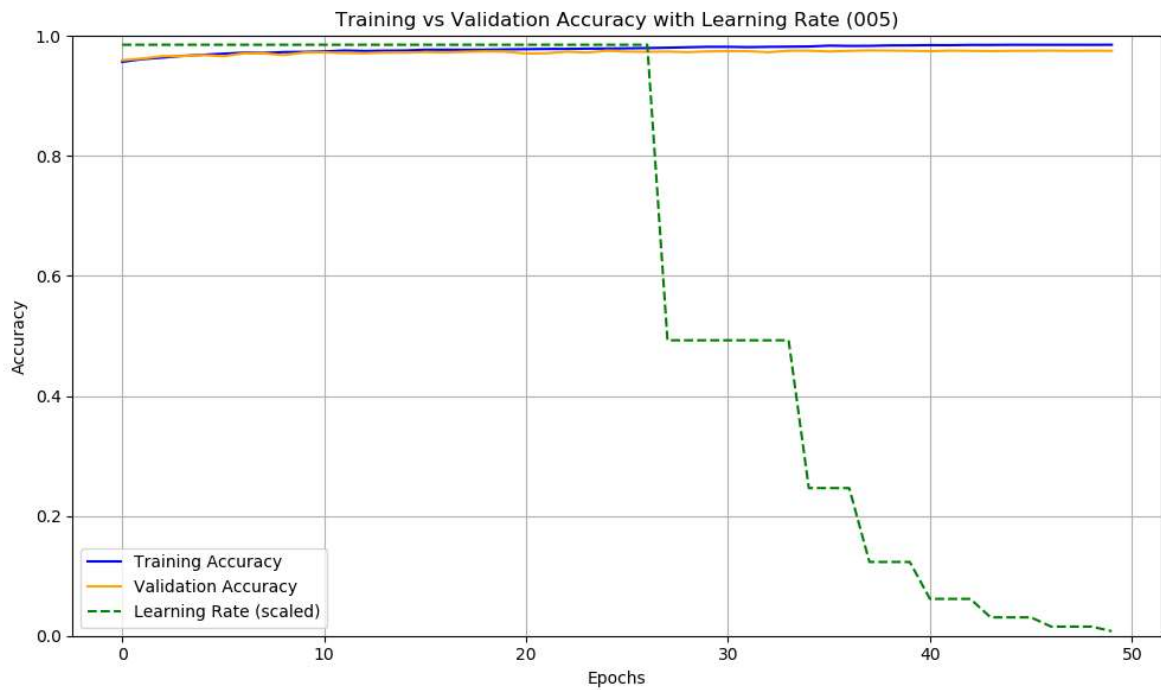


Figure 33: Model 005 Training Accuracy, Validation Accuracy, Learning Rate (scaled)

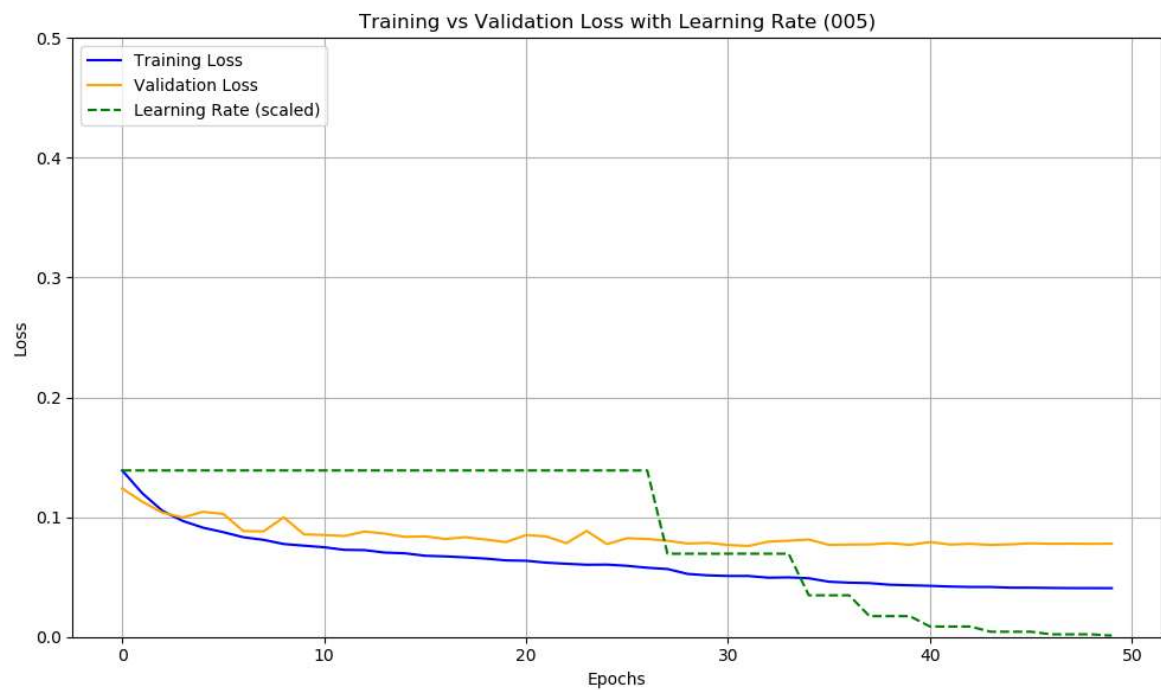


Figure 34: Model 005 Training Loss, Validation Loss, Learning Rate (scaled)

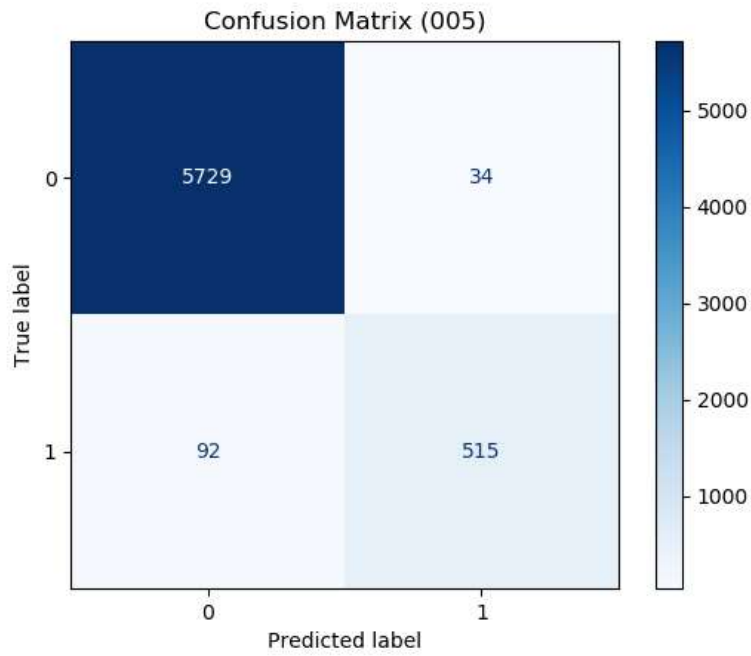


Figure 35: Model 005 Confusion Matrix

Classification Report (005)				
-	Precision	Recall	F1-Score	Support
0	0.98	0.99	0.99	5,763
1	0.94	0.85	0.89	607
Accuracy	-	-	0.98	6,370
Macro Average	0.96	0.92	0.94	6,370
Weighted Average	0.98	0.98	0.98	6,370

Figure 36: Model 005 Classification Report

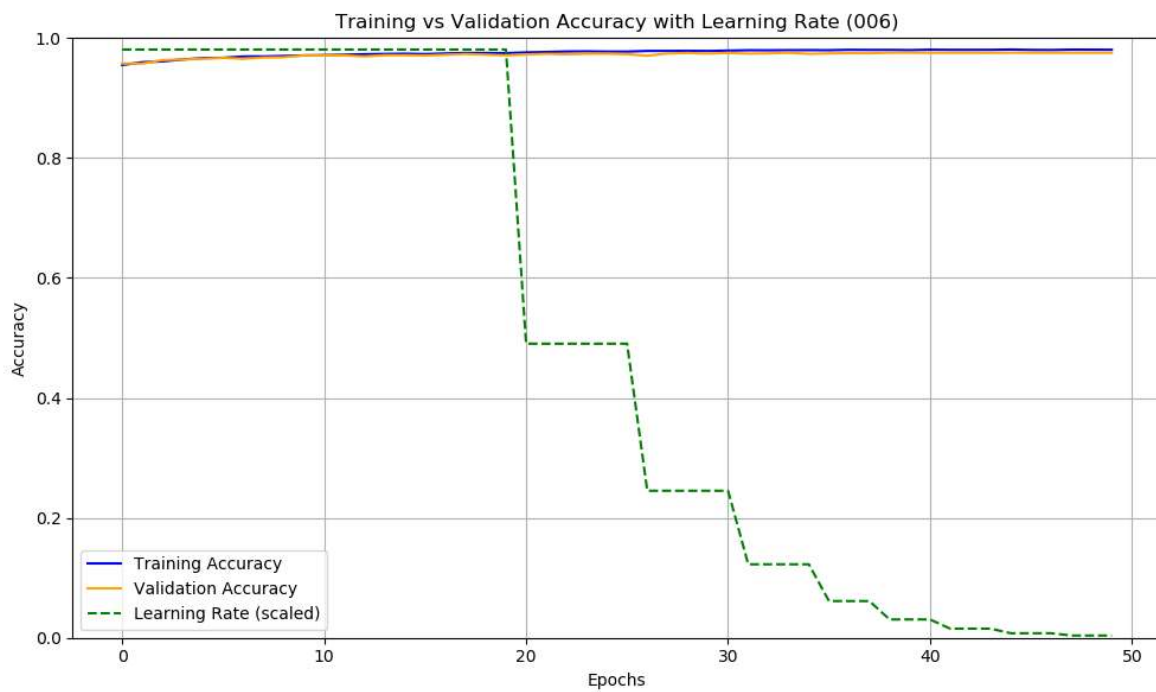


Figure 37: Model 006 Training Accuracy, Validation Accuracy, Learning Rate (scaled)

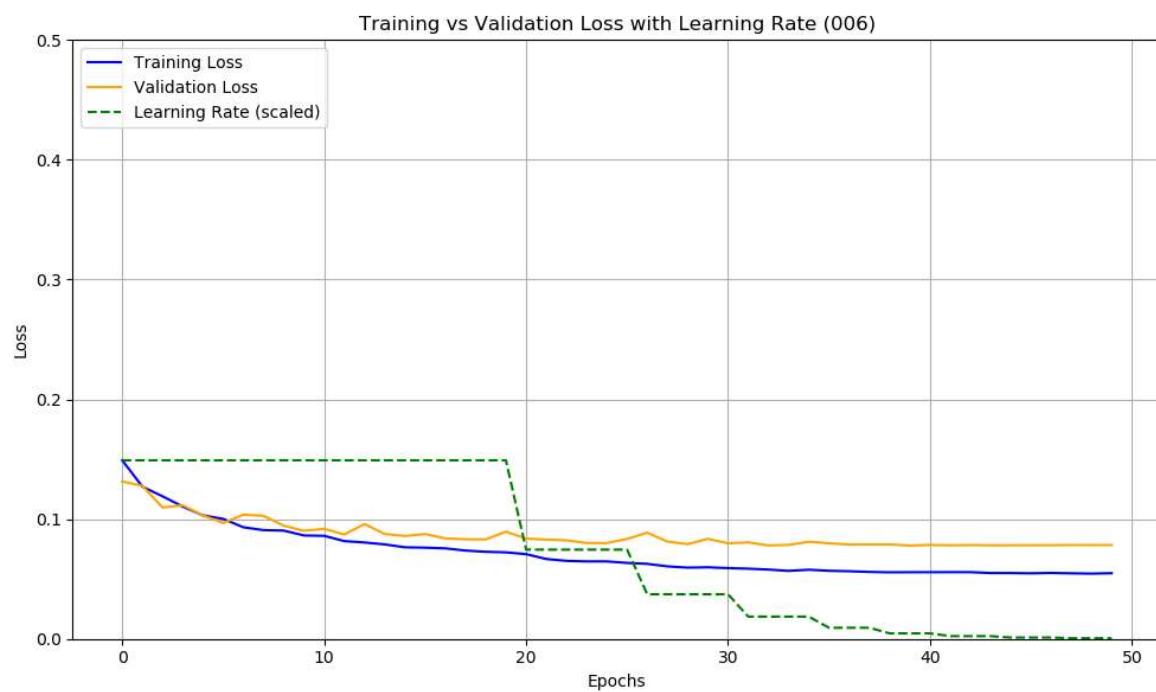


Figure 38: Model 006 Training Loss, Validation Loss, Learning Rate (scaled)

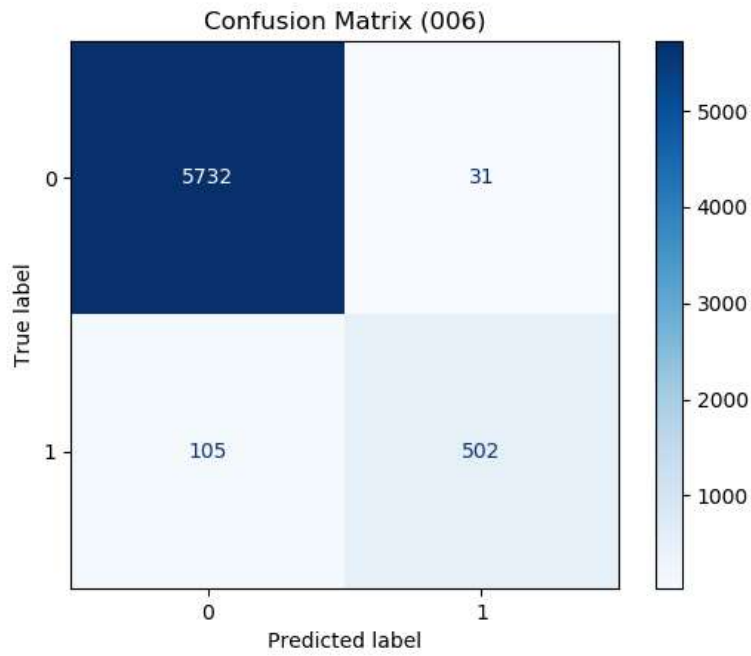


Figure 39: Model 006 Confusion Matrix

Classification Report (006)				
-	Precision	Recall	F1-Score	Support
0	0.98	0.99	0.99	5,763
1	0.94	0.83	0.88	607
Accuracy	-	-	0.98	6,370
Macro Average	0.96	0.91	0.93	6,370
Weighted Average	0.98	0.98	0.98	6,370

Figure 40: Model 006 Classification Report

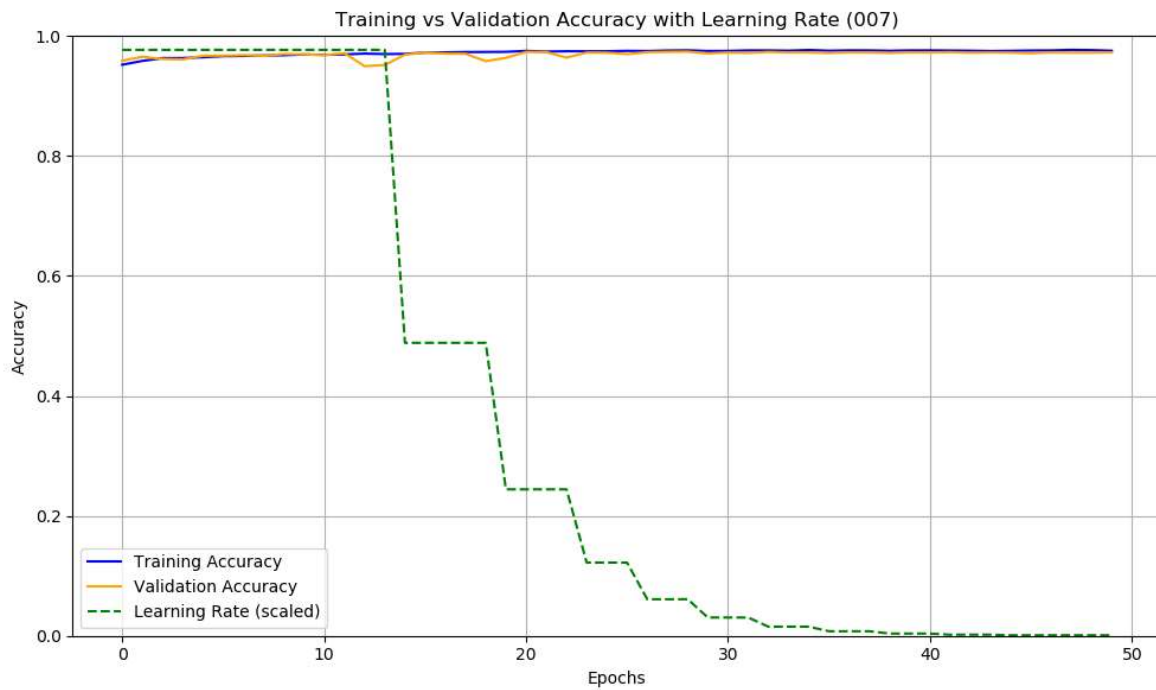


Figure 41: Model 007 Training Accuracy, Validation Accuracy, Learning Rate (scaled)

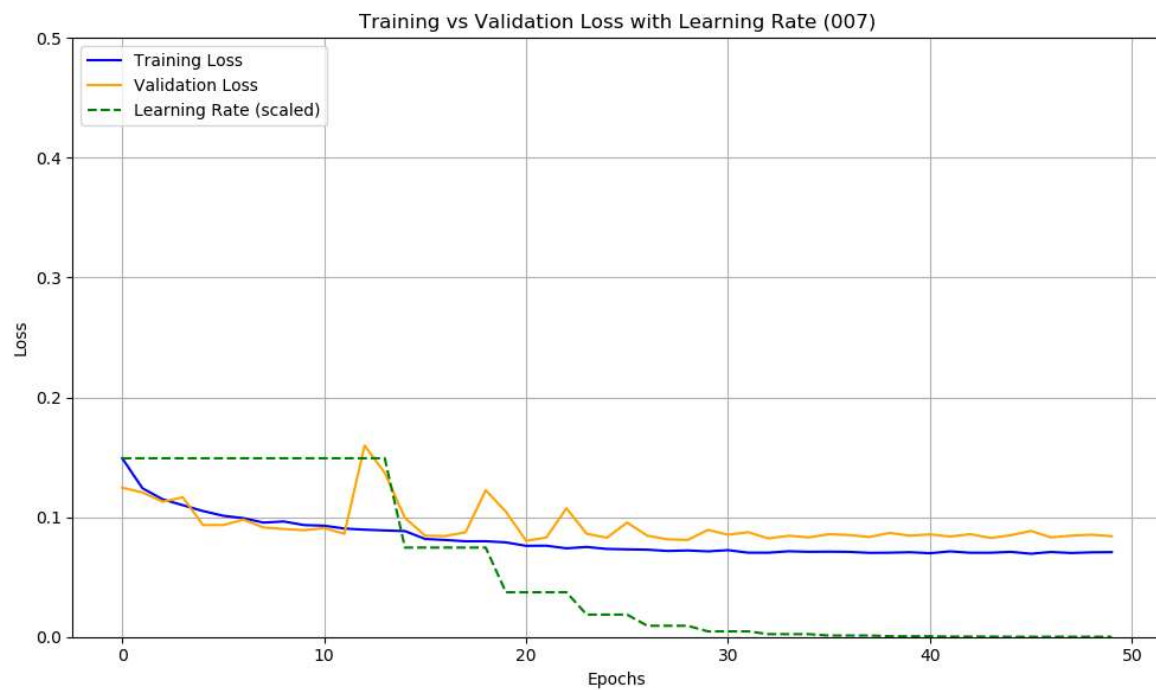


Figure 42: Model 007 Training Loss, Validation Loss, Learning Rate (scaled)

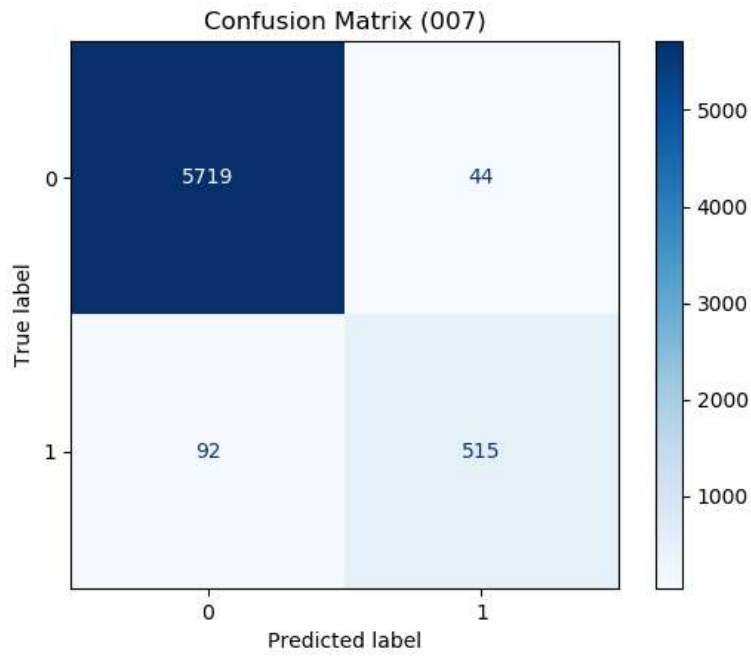


Figure 43: Model 007 Confusion Matrix

Classification Report (007)				
-	Precision	Recall	F1-Score	Support
0	0.98	0.99	0.99	5,763
1	0.92	0.85	0.88	607
Accuracy	-	-	0.98	6,370
Macro Average	0.95	0.92	0.94	6,370
Weighted Average	0.98	0.98	0.98	6,370

Figure 44: Model 007 Classification Report